

BP-DeepONet: A new method for cuffless blood pressure estimation using the physics-informed DeepONet

Lingfeng Li^a, Xue-Cheng Tai^{b,*}, Raymond Hon-Fu Chan^{a,c}

^a*Hong Kong Centre for Cerebro-cardiovascular Health Engineering, Hong Kong, China*

^b*Norwegian Research Center, Bergen, Norway*

^c*Department of mathematics, City university of Hong Kong, Hong Kong, China*

Abstract

Cardiovascular diseases (CVDs) are the leading cause of death worldwide, with blood pressure serving as a crucial indicator. Arterial blood pressure (ABP) waveforms provide continuous pressure measurements throughout the cardiac cycle and offer valuable diagnostic insights. Consequently, there is a significant demand for non-invasive and cuff-less methods to measure ABP waveforms continuously. Accurate prediction of ABP waveforms can also improve the estimation of mean blood pressure, an essential cardiovascular health characteristic. This study proposes a novel framework based on the physics-informed DeepONet approach to predict ABP waveforms. Unlike previous methods, our approach requires the predicted ABP waveforms to satisfy the **Navier-Stokes equation** with a time-periodic condition and a Windkessel boundary condition. Notably, our framework is the first to predict ABP waveforms continuously, both with location and time, within the part of the artery that is being simulated. Furthermore, our method only requires ground truth data at the outlet boundary and can handle periodic conditions with varying periods. Incorporating the Windkessel boundary condition in our solution allows for generating natural physical reflection waves, which closely resemble measurements observed in real-world cases. Moreover, accurately estimating the hyper-parameters in the Navier-Stokes equation for our simulations poses a significant challenge. To overcome this obstacle, we introduce the concept of meta-learning, enabling the neural networks to learn these parameters during the training process. To demonstrate the effectiveness of our approach, we conduct numerical experiments that showcase its superiority over traditional methods, which typically only predict systolic blood pressure (SBP) and diastolic blood pressure (DBP). Our proposed method not only accurately predicts continuous ABP waveforms but also outperforms these traditional approaches.

1. Introduction

According to the World Health Organization (WHO), cardiovascular diseases (CVDs) are the leading cause of death globally [1]. In 2019, there were about 32% of all global deaths caused by CVDs. The majority of these deaths were due to heart attack and stroke. There is sufficient evidence to support that high blood pressure (or hypertension) is the predominant cause of CVDs [2]. In practice, the morphology of the arterial blood pressure (ABP) waveforms can reflect the cardiovascular status of humans [3–6]. Therefore, it is of great importance to monitor ABP waveforms continuously to detect and prevent diseases.

In the clinical setting, the gold standard for ABP waveform monitoring is the invasive method, which involves implanting an invasive pressure sensor into the artery. The invasive method is very risky, and the used devices are expensive. It is commonly used in intensive care units (ICUs). On the other hand, the non-invasive method is a more desirable choice for the daily monitoring of ABP waveforms.

The deep learning-based method for non-invasive ABP waveform estimation has attracted huge research interest in recent years. These methods usually estimate the ABP from other physiological signals that are much easier to measure. Electrocardiogram (ECG) and photoplethysmogram (PPG) are the most commonly used signals. The ECG signal records the heart's electrical activity through cardiac cycles. It is measured by some electrodes, which can

*Corresponding author

Email addresses: lfli@hkcoche.org (Lingfeng Li), xtai@norceresearch.no (Xue-Cheng Tai), raymond.chan@cityu.edu.hk (Raymond Hon-Fu Chan)

detect the small electrical changes resulting from the cardiac activity. The patterns of ECG signals can indicate many cardiac abnormalities. The PPG signal is an optical signal that can detect blood volume changes in the microvascular bed of tissue. It is often obtained by illuminating the skin with a pulse oximeter and measuring changes in light absorption. Examples of filtered ECG and PPG signals are shown in Figure 1.

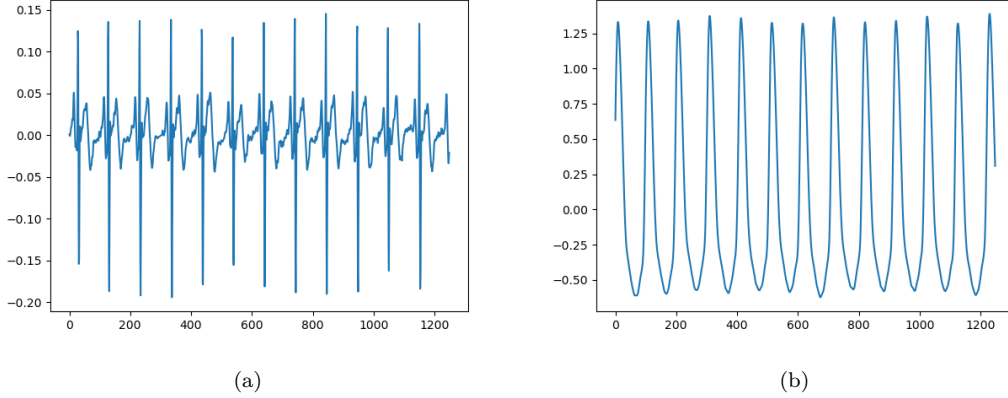


Figure 1: Filtered ECG and PPG signals in 10 seconds.

Deep learning models were introduced to estimate systolic and diastolic blood pressure in [7]. Then, more complicated neural networks are used to estimate the entire ABP waveform [8–10]. In [8], the authors proposed a neural network consisting of a convolutional neural network (CNN), a bi-directional gated recurrent unit (Bi-GRU), and attention layers to estimate ABP from ECG and PPG. In [9], a network that predicts ABP only using PPG signals was proposed. In [10], the authors introduced a modified transformer-based model to predict the ABP waveform from ECG, PPG, and some morphological extracted features.

The models above are all trained in a data-driven way, i.e., trained by minimizing the difference between the ground truth label and the network prediction. Since the labeled data is usually measured by invasive methods in the radial artery [11], the trained neural network will only predict the radial ABP waveform. However, ABP waveforms differ at different locations in the arterial systems [12, 13] (See Figure 2), and radial waveforms may not be sufficient to diagnose CVD and reflect the status of the cardiovascular system. For example, some studies have shown that the brachial waveform is more valuable than the radial waveform when assessing cardiac contractility [14, 15]. Another important observation is the pulse pressure amplification effect. Pulse pressure is defined as the difference between the maximum and minimum of the pressure waveforms, and this difference will increase through the propagation of the pulse wave in the arterial system. All current deep learning-based algorithms can only predict radial ABP waveforms since they are trained with labeled data measured at the radial artery.

In this work, we aim to develop new methods to predict ABP waveforms at different locations and at all times in the artery conditioned to the physical law that governs the blood flow, i.e. the Navier-Stokes equation. We use the physics-informed DeepONet (PI-DeepONet) [16]. DeepONet was initially proposed to learn the solution operator that maps variable inputs to the corresponding latent solutions of certain partial differential equation (PDE) systems [17], and its training process is entirely data-driven. If the underlying PDE system is explicitly given, then the PDE can be incorporated into the training loss like the physics-informed neural networks (PINN) [18]. The DeepONet trained in this way is called the physics-informed DeepONet. In the arterial system, the hemodynamics, i.e., blood flow dynamics, can be modeled and simulated by the Navier-Stokes equation [19–21]. Therefore, we can incorporate the Navier-Stokes equation into the training loss and learn the solution operator that maps physiological signals to solutions of the corresponding PDEs. In our setting, the spatial domain of the Navier-Stokes equation is a segment of the artery, and the labeled data can serve as a Dirichlet boundary condition. After providing other necessary initial/boundary conditions, we can identify the PDE solution in the whole domain. Consequently, we can obtain the ABP waveforms at different locations and at all times.

We summarize our main contributions as follows:

- This work is the first attempt to apply physics-informed operator learning to ABP waveform estimation. The proposed methods are the first ones to predict ABP waveforms at different locations in the artery conditioned

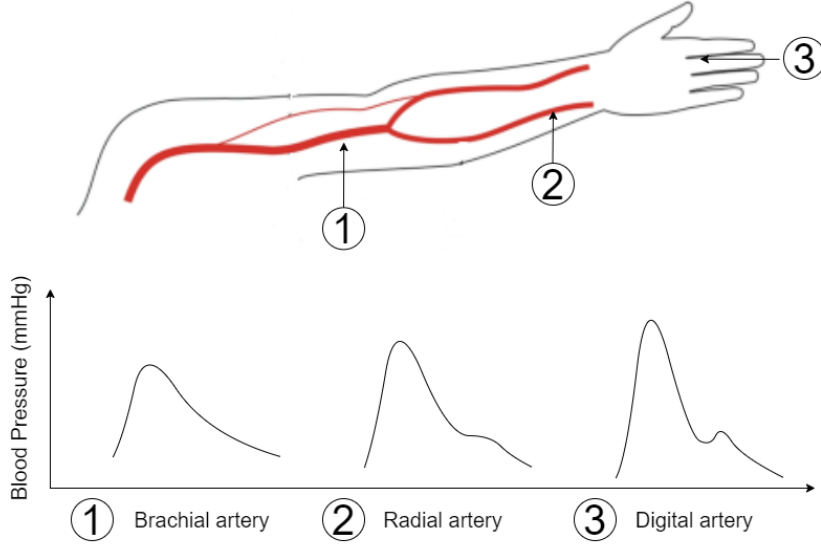


Figure 2: Illustration of the pulse pressure amplification in arterial systems.

by the physical law that governs the blood flow, i.e., the Navier-Stokes equation. The proposed methods enforce neural network solutions to satisfy a 1-D Navier-Stokes equation with a Windkessel boundary condition and a time-periodic condition. The Windkessel boundary condition can simulate physical reflection waves.

- The proposed methods can predict the blood pressure waveforms with reasonably good accuracy and preserve some physical properties like pulse pressure amplification.
- To the best of our knowledge, the proposed methods are the first ones that incorporate variable time-periodic conditions to DeepONet. We also propose an effective implementation method for the network training.
- We also incorporate meta-learning into the training to estimate model hyper-parameters automatically from the input physiological signals.
- During training, we only require the blood pressure measurement at the outlet side of the domain, and no extra measurement is needed.

2. One-dimensional model for blood flow simulation

The hemodynamics in a vessel segment can be effectively modeled and simulated by the one-dimensional Navier-Stokes equation [19, 20]. The one-dimensional model is a simplification of the three-dimensional Navier-Stokes equation by making several assumptions on the computational domain:

- The vessel is a straight cylinder with the axis oriented along the coordinate z direction (See Figure 3).
- The vessel wall displaces along the radial direction only, which implies that each axial cross-section is always circular.
- The blood pressure and blood flow rate are constant within each axial cross-section.
- All body forces, like the gravity, are neglected.

Thus, the hemodynamics can be described by three quantities, namely the blood flow rate $Q(z, t)$, the axial cross-sectional area $A(z, t)$, and the blood pressure $P(z, t)$, where $z \in [0, L]$ is the spatial coordinates and $t \in [0, T]$ is

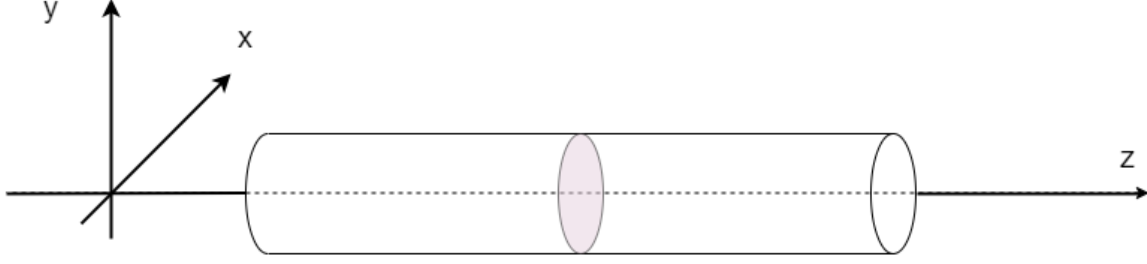


Figure 3: Simplification of the three-dimensional model to one-dimensional model. The vessel is assumed to be a straight cylinder, and each axial cross-section is circular.

the temporal coordinates. L denotes the length of the vessel segment, and T represents the time duration. Then, the one-dimensional model can be written as:

$$\frac{\partial A}{\partial t} + \frac{\partial Q}{\partial z} = 0, \quad (1)$$

$$\frac{\partial Q}{\partial t} + \frac{\partial}{\partial z} \left(\frac{Q^2}{A} \right) + \frac{A}{\rho} \frac{\partial P}{\partial z} + K_r \frac{Q}{A} = 0, \quad (2)$$

where ρ is the blood density, and K_r is a resistance parameter related to blood viscosity. To close this system, the blood pressure P and the cross-sectional area are assumed to satisfy:

$$P = P_{ext} + \beta \frac{\sqrt{A} - \sqrt{A_0}}{A_0}, \quad (3)$$

where P_{ext} is the external pressure, A_0 is the cross-sectional area at the reference state, and β is a constant depending on the physical and mechanical properties of vessels. More details of the model derivation can be found in [19, 20]. A simple characteristic analysis shows that the forward characteristic variable W_1 and backward characteristic variable W_2 in the system (1)-(2) are:

$$W_1(A, Q) = \frac{Q}{A} + 4\sqrt{\frac{\beta}{2\rho A_0}}(A^{1/4} - A_0^{1/4}),$$

and

$$W_2(A, Q) = \frac{Q}{A} - 4\sqrt{\frac{\beta}{2\rho A_0}}(A^{1/4} - A_0^{1/4}).$$

The forward wave and backward wave are traveling at the speed

$$\lambda_1(A, Q) = \frac{Q}{A} + \sqrt{\frac{\beta}{2\rho A_0}} A^{1/4}$$

and

$$\lambda_2(A, Q) = \frac{Q}{A} - \sqrt{\frac{\beta}{2\rho A_0}} A^{1/4}$$

respectively. In practice, we have

$$\sqrt{\frac{\beta}{2\rho A_0}} A^{1/4} \gg Q/A,$$

so W_1 and W_2 always travel in opposite directions. The speed of the forward wave $\lambda_1 \approx \sqrt{\frac{\beta}{2\rho A_0}} A^{1/4}$ is often called pulse wave velocity.

Using the relation (3), we can rewrite the equations (1)-(2) as

$$\frac{\partial U}{\partial t} + H(U) \frac{\partial U}{\partial z} = B(U), \quad (z, t) \in [0, L] \times [0, T] \quad (4)$$

where

$$U = \begin{pmatrix} P \\ Q \end{pmatrix},$$

$$H(U) = \begin{pmatrix} 0 & \frac{\beta}{2a(P)A_0} \\ \frac{a(P)^2}{\rho} - \frac{2A_0Q^2}{\beta a^3(P)} & \frac{2Q}{a(P)^2} \end{pmatrix},$$

$$B(U) = \begin{pmatrix} 0 \\ -K_r \frac{Q}{a(P)^2} \end{pmatrix},$$

and

$$a(P) = \frac{A_0}{\beta} (P - P_{ext}) + \sqrt{A_0}.$$

Even though the one-dimensional model can not capture the blood flow details, it can effectively describe the wave propagation within the vessel. This model has been validated by both *in vitro* data [22, 23] and *in vivo* data [24, 25]. There are many effective numerical solvers, like the MacCormack scheme [26, 27], Taylor-Galerkin scheme [28, 29], MUSCL [30, 31], and the local discontinuous Galerkin scheme [32–34]. A comparison of different numerical methods has also been done in [35].

3. Physics-informed machine learning for PDEs

Machine learning algorithms for PDEs have been extensively studied in recent years. In general, there are two main directions in this research field: learning an instance of PDEs and learning the solution operator of PDEs. The most famous work along the first direction is the physics-informed neural networks (PINNs) proposed in [18]. This work aims to solve an instance of PDEs using a neural network. Compared to traditional numerical solvers, the PINN is mesh-free, easy to implement, and can solve high-dimensional problems. However, it can not achieve very high accuracy for non-linear PDEs. The second direction aims to learn the solution operator that maps from some input spaces to the solution space using some neural networks. This type of network is also called the neural operator. The most popular neural operators include the DeepONet [17] and the Fourier Neural operator (FNO) [36]. The learned network can quickly predict the PDE solution corresponding to the given input. We will provide a general introduction to two types of methods in this section.

3.1. Physics-informed neural networks (PINNs)

Let's consider a general form of a d -dimensional boundary value problem:

$$\begin{aligned} \mathcal{L}(u)(x) &= f(x), & x \in \Omega \\ \mathcal{B}(u)(x) &= g(x), & x \in \partial\Omega \end{aligned}$$

where $\Omega \in \mathbb{R}^d$ is the problem domain, $\mathcal{L}(u)$ denotes a general differential operator, and $\mathcal{B}(u)$ denotes corresponding boundary conditions. The PINN method first defines a neural network to approximate the solution to this specific instance of PDE: $\mathcal{N}(x; \theta) : \Omega \rightarrow \mathbb{R}$, where θ denotes the set of all trainable parameters in the network. Then, this network is trained by solving the optimization problem:

$$\min_{\theta} \sum_{j=1}^{N_0} |\mathcal{L}(\mathcal{N}(x_j; \theta)) - f(x_j)|^2 + \omega \sum_{j=1}^{N_1} |\mathcal{B}(\hat{x}_j) - g(\hat{x}_j)|^2$$

where ω is the weight of the boundary residual, $\{x_j\}_{j=1}^{N_0}$ is collocation points randomly sampled from Ω , and $\{\hat{x}_j\}_{j=1}^{N_1}$ is collocation points randomly generated from $\partial\Omega$. The partial derivatives of $\mathcal{N}$ can be easily calculated through

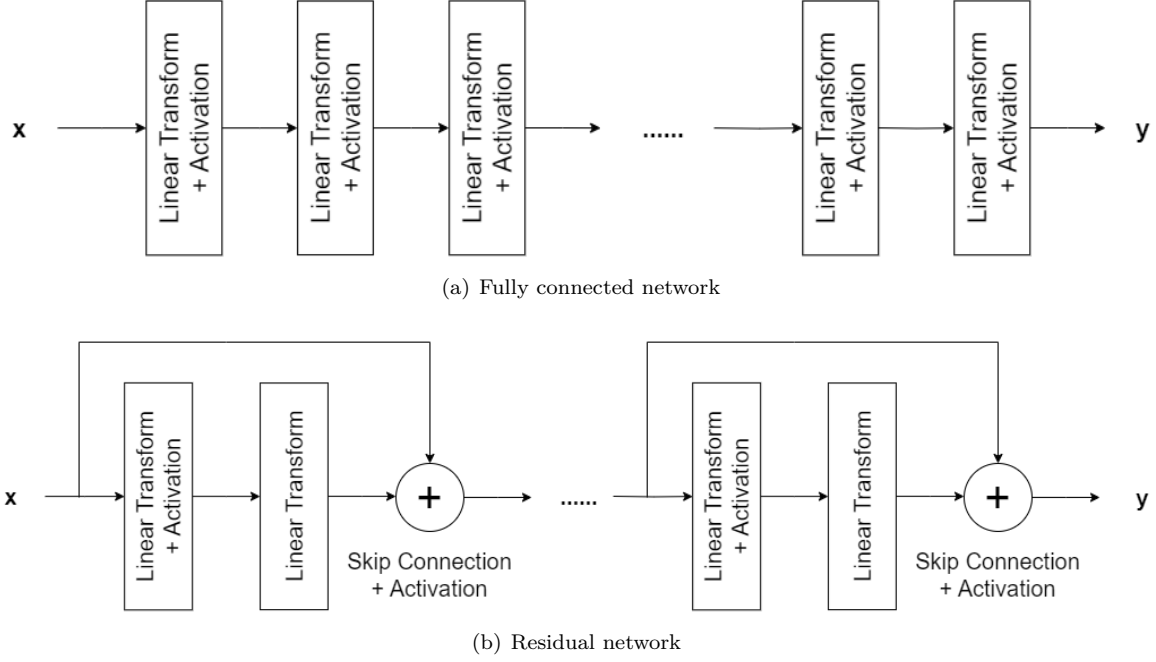


Figure 4: Structures of the fully connected network and the residual network.

the back-propagation mechanism. An efficient implementation is provided in the DeepXDE package [37] for both PyTorch and TensorFlow platforms. Some similar algorithms in the literature use similar ideas, such as the deep Ritz method (DRM) [38] and the mixed residual method (MIM) [39, 40]. The DRM uses the variational formulation of PDEs to train networks, and the MIM transforms high-order PDEs into first-order systems to train networks. Various numerical analyses have also been conducted for these physics-informed methods [41–45].

One popular network structure for the PINN method is the fully connected network (FCN) (Figure 4(a)). An FCN is typically defined as compositions of many fully connected layers:

$$\begin{aligned}
\mathcal{N}_{FCN}(x; \theta) &:= (\mathcal{F}_L \circ \mathcal{F}_{L-1} \circ \cdots \circ \mathcal{F}_1)(x), \\
\mathcal{F}_l(z) &:= \sigma_l(W_l z + b_l), \quad \forall l = 1, \dots, L-1, \\
W_l &\in \mathbb{R}^{d_{l,in} \times d_{l,out}}, \quad b_l \in \mathbb{R}^{d_{l,out}}, \\
d_{1,in} &= d, \quad d_{l,in} = d_{l-1,out}, \quad l = 2, \dots, L-1, \\
\mathcal{F}_L(z) &:= \sigma_L(W_L z + b_L), \quad W_L \in \mathbb{R}^{d_{L-1,out}, 1}, \quad b_L \in \mathbb{R},
\end{aligned}$$

where σ_l is the activation function for each layer, $d_{l,in}$ denotes the input dimension of the l th layer, and $d_{l,out}$ denotes the output dimension of the l th layer. Choices for the activation function usually include Sigmoid, Tanh, ReLU, etc. For the last layer $\mathcal{F}_L$, the activation function can be the identity function, i.e., $\sigma_L(z) = z$. When increasing the depth L , the training of the FCN may become slower because of the vanishing gradient effect. To overcome this limitation, we can use the residual network incorporating the skip connection structure [46]. A residual network (Figure 4(b)) is defined as:

$$\begin{aligned}
\mathcal{N}_{Res}(x; \theta) &:= (\hat{\mathcal{F}}_L \circ \hat{\mathcal{F}}_{L-1} \circ \cdots \circ \hat{\mathcal{F}}_1)(x), \\
\hat{\mathcal{F}}_l(z) &:= \sigma_l(W_{l,2} \sigma_l(W_{l,1} z + b_{l,1}) + b_{l,2} + z), \quad \forall l = 1, \dots, L-1, \\
W_{l,1} &\in \mathbb{R}^{d_{l,in} \times d_{l,out}}, \quad W_{l,2} \in \mathbb{R}^{d_{l,out} \times d_{l,out}}, \quad b_{l,1}, b_{l,2} \in \mathbb{R}^{d_{l,out}}, \\
d_{1,in} &= d, \quad d_{l,in} = d_{l-1,out}, \quad l = 2, \dots, L-1, \\
\mathcal{F}_L(z) &:= \sigma_L(W_L z + b_L), \quad W_L \in \mathbb{R}^{d_{L-1,out}, 1}, \quad b_L \in \mathbb{R}.
\end{aligned}$$

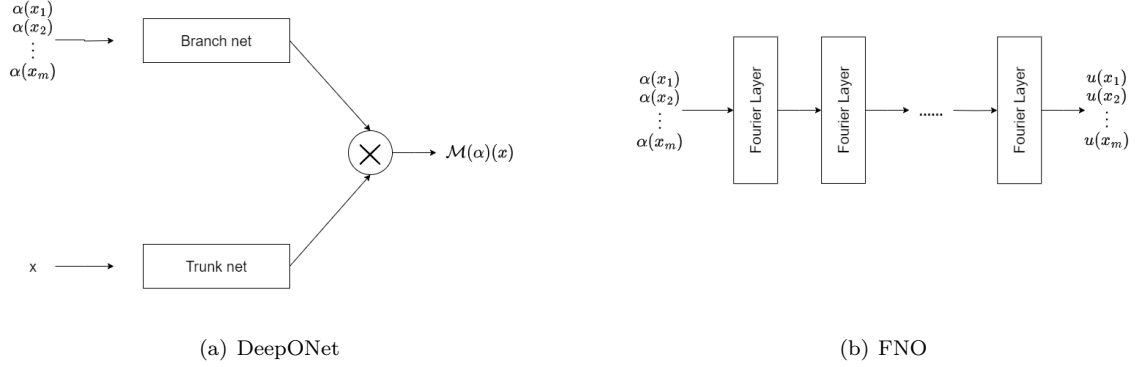


Figure 5: Structures of the DeepONet and FNO.

3.2. Neural operators for PDEs

Let's consider a class of parametric PDEs:

$$\begin{aligned}\mathcal{L}(u)(x; \alpha) &= f(x; \alpha), \quad x \in \Omega \\ \mathcal{B}(u; \alpha)(x) &= g(x; \alpha), \quad x \in \partial\Omega,\end{aligned}$$

where α is the parameters that define the problem; for example, α can be the diffusion coefficient in a convection-diffusion equation. The neural operator aims to approximate the solution operator $\mathcal{M} : \mathcal{A} \rightarrow \mathcal{U}$, where $\mathcal{A}$ is the function space for α , and $\mathcal{U}$ is the space for the solution u . In practice, the input space can also be other function spaces, e.g., the space of the source term $f(x)$. Two types of neural operators have been extensively applied to various tasks in the literature: the DeepONet [17] and the FNO [36].

A plain DeepONet (Figure 5(a)) consists of two sub-networks: a branch net and a trunk net. It is defined as

$$\mathcal{N}_{DON}(\hat{\alpha}, x) := \text{Branch}(\hat{\alpha})^\top \text{Trunk}(x)$$

where $\hat{\alpha}$ is the discretization of α on a given mesh, x is any point in Ω . The outputs of the branch net and the trunk net are p -dimensional vectors, where p is a user-specified hyper-parameter. The DeepONet $\mathcal{N}(\hat{\alpha}, x)$ gives a prediction to $\mathcal{M}(\alpha)(x)$. To predict the solution on multiple points, we need to evaluate the trunk net multiple times.

A plain FNO (Figure 5(b)) is the composition of many Fourier layers:

$$\begin{aligned}\mathcal{N}_{FNO}(\hat{\alpha}) &:= (\tilde{\mathcal{F}}_L \circ \tilde{\mathcal{F}}_{L-1} \cdots \circ \tilde{\mathcal{F}}_1)(\hat{\alpha}) \\ \tilde{\mathcal{F}}_l(z) &:= \sigma_l(W_l z + FFT^{-1}(R_l(FFT(z))))\end{aligned}$$

where σ_l is the activation function, FFT is the fast Fourier transformation, FFT^{-1} is the inverse fast Fourier transformation, W_l is a linear transformation, and $R_l(z)$ is an operator that includes a truncation of first m frequency modes and mode-wise complex matrix multiplications. The output of the FNO is of the same size as $\hat{\alpha}$.

The training of neural operators is fully data-driven. Suppose we have a set of $\{\hat{\alpha}_i\}_{i=1}^N$ from $\mathcal{A}$ and corresponding PDE solutions $\{\hat{u}_i\}_{i=1}^N$ defined on a discretization $\{x_j\}_{j=1}^M$. Then we train the neural operator by minimizing the L_2 difference between the network outputs and the reference solution:

$$\min_{\theta} \frac{1}{MN} \sum_{i=1}^N \sum_{j=1}^M |\mathcal{N}_{DON}(\hat{\alpha}_i, x_j) - \hat{u}_i(x_j)|^2$$

for the DeepONet, or

$$\min_{\theta} \frac{1}{N} \sum_{i=1}^N \|\mathcal{N}_{FNO}(\hat{\alpha}_i) - \hat{u}_i\|_2^2$$

for the FNO. A comprehensive study of two different neural operators is given in [47], which includes numerical experiments on various linear and non-linear PDEs. In general, the performances of two types of neural operators are comparable in various problems. The FNO is more efficient in training and testing, while the DeepONet has a more flexible structure that can be easily applied to different tasks.

Besides the data-driven training method, we can apply the physics-informed loss like PINN to train neural operators. Such methods are called physics-informed neural operators [16, 48, 49]. In many engineering problems, obtaining the entire solution to PDEs may be very difficult or expensive. Consequently, it is more desirable to incorporate the physics laws into the training process to regularize the neural operators. We can construct a similar residual loss as the PINN method for each pair of training samples and minimize the summation over all samples. For the DeepONet, we can use back-propagation to evaluate the derivatives. For the FNO, we can use numerical differentiation to evaluate the derivatives.

4. The physics informed DeepONet for ABP waveform estimation

4.1. Problem setup

This work aims to learn the operator that maps the physiological signals to the solution of the Navier-Stokes equation:

$$\mathcal{O} : \mathcal{S}([0, T]) \rightarrow \mathcal{U}([0, L] \times [0, T])$$

where $\mathcal{S}([0, T])$ is the set of physiological signals of length T and $\mathcal{U}([0, L] \times [0, T])$ is the space of solutions to Navier-Stokes equations defined on $[0, L] \times [0, T]$.

Suppose we have a training dataset consisting of N pairs of samples:

$$\{(s_i, p_i)\}_{i=1}^N,$$

where s_i is a set of physiological signals, and p_i is the corresponding ABP waveform measured in the radial artery that lies in the forearm. All waveform data have the same length. The physiological signal s_i serves as the input to the neural network and the blood pressure waveform p_i is used to define the boundary condition.

For each sample $i = 1, \dots, N$, its hemodynamics is defined by $U_i = (P_i, Q_i)$. We assume each U_i satisfies the Navier-stokes system described in (4):

$$\frac{\partial U_i}{\partial t} + H(U_i) \frac{\partial U_i}{\partial z} = B(U_i), \quad (z, t) \in [0, L] \times [0, T]. \quad (5)$$

We also need to provide suitable initial and boundary conditions to identify unique PDE solutions. When solving this system using some traditional numerical methods, we usually need to impose an inlet boundary condition and an outlet boundary condition. The inlet boundary condition is usually set as a Dirichlet boundary condition which prescribes the value of the blood pressure P or blood flow rate Q at the inlet side. The outlet boundary condition can determine the reflection waves that run backward in the artery. A simple choice for the outlet boundary condition is the non-reflecting boundary condition [20]: $W_2 = 0$ where W_2 is the backward characteristic variable. Though the non-reflecting boundary condition is very easy to implement, it can not provide a physically correct reflecting wave. A better choice is the three-element Windkessel boundary condition [50]:

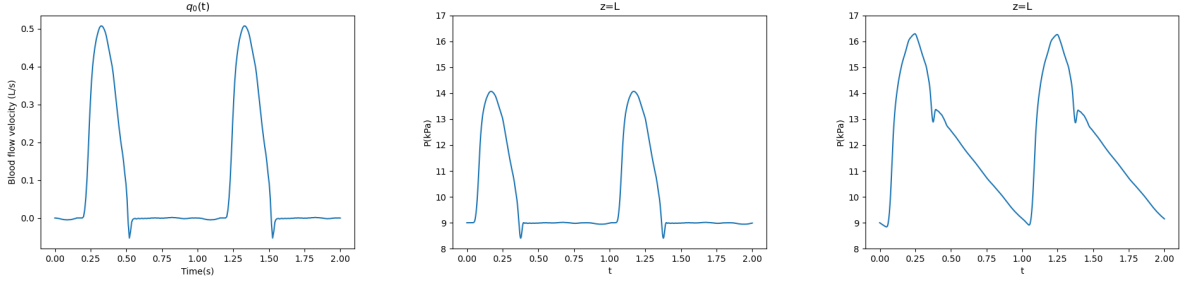
$$W(P_i, Q_i) := Q_i(L, t) \left(1 + \frac{R_1}{R_2}\right) + CR_1 \frac{\partial Q_i}{\partial t}(L, t) - \frac{P_i(L, t)}{R_2} - C \frac{\partial P_i}{\partial t}(L, t) = 0, \quad t \in [0, T]. \quad (6)$$

where R_1 , R_2 , and C are some hyper-parameters related to the artery properties.

Remark. The Windkessel model describes the hemodynamics of the arterial system in terms of resistance and compliance, and it is widely used as an outflow boundary condition in the numerical simulation of human arterial systems [21, 22, 51]. The Navier-Stokes equation with this Windkessel boundary condition can produce physical reflection waves in the solutions. A simple comparison of two outflow boundary conditions is shown in Figure 6 where we can observe that the blood pressure simulated using the non-reflecting boundary condition is not physically correct.

In our case, since the pressure data p_i , $i = 1, \dots, N$, is measured near the end of radial arteries, it is better to formulate it also as an outlet boundary condition:

$$P_i(L, t) = p_i(t), \quad t \in [0, T]. \quad (7)$$



(a) Prescribed inlet boundary condition (b) Simulated blood pressure using the non-reflecting boundary condition (c) Simulated blood pressure using the Windkessel boundary condition

Figure 6: Comparison of the simulated blood pressure using different outflow boundary conditions. (a) is the inlet boundary condition given at $z = 0$; (b) is the simulated blood pressure at $z = L$ using the non-reflecting outflow boundary condition; and (c) is the simulated blood pressure at $z = L$ using the 3-element Windkessel outflow boundary condition. (b) and (c) are computed using a standard MacCormack scheme.

To better simulate the reflection waves in the artery, we use the three-element Windkessel model (6) as another boundary condition.

For the initial condition, we impose a time-periodic condition on both P_i and Q_i instead of giving a prescribed initial value:

$$U_i(z, t + \delta_i) = U_i(z, t), \quad (z, t) \in [0, L] \times [0, T] \quad (8)$$

where δ_i is the period for the i -th training sample, and it can be estimated from the physiological signals s_i . Typically, the waveforms are almost periodic within a short time.

Remark. Conventional methods directly prescribe the initial value of solutions. One commonly used initial condition is $U(z, 0) = (P_{ext}, 0)$. However, it is very difficult to measure the value of P_i and Q_i at the initial time for each sample and it is not realistic to assume an initial condition like $U(z, 0) = (P_{ext}, 0)$. Therefore, using a time-periodic condition is more suitable in our situation.

By solving the system (4) with the boundary condition (7) (6) and initial condition (8), we would be able to predict the hemodynamics in the radial artery. For conventional numerical methods, it would be difficult to solve under this setting, because there is no prescribed initial value given and there is no boundary condition at the inlet side; but it is easy to handle by the physics-informed method.

In this problem, the period for each sample is different, which will cause some difficulty in computing the loss function. In this work, we will propose a new efficient implementation method to handle this issue.

4.2. BP-DeepONet

We propose a new physics-informed DeepONet, BP-DeepONet, to learn the operator (4.1). We adopt the same structure as the original DeepONet. For the branch net, we use a structure that combines a one-dimensional convolutional ResNet and a bi-directional LSTM, see Figure 7. The inputs to the branch net are some segments of physiological signals s_i and the output would be two latent vectors $\mathbf{b}_1$ and $\mathbf{b}_2$. $\text{conv1d}(\text{channels}=c, \text{kernel}=s)$ represents a standard one-dimensional convolutional layer whose number of output channels is c and kernel size is s . Bi-LSTM ($\text{size}=h, \text{layers}=l$) represents a bi-directional LSTM block whose input size and hidden size are h , and the number of layers is l . A similar structure has already been used in [52] to extract features from physiological signals to predict ABP waveforms. For the trunk net, we adopt a simple, fully connected ResNet as Figure 4(b) with the feature expansion (Figure 8). Feature expansion [53] is a technique to impose the periodic condition on PINN. To impose the time-periodic constraints (8), we can simply extend the inputs (z, t) to

$$(z, \sin(\frac{2\pi t}{\delta_i}), \cos(\frac{2\pi t}{\delta_i}), \sin(\frac{4\pi t}{\delta_i}), \cos(\frac{4\pi t}{\delta_i}), \dots).$$

Then, our network outputs will be strictly time-periodic with period δ_i . The output of the trunk net is a vector $\mathbf{k}$ whose dimension is the same as $\mathbf{b}_1$ and $\mathbf{b}_2$. The final prediction of the BP-DeepONet will be calculated as follows:

$$P_\theta(s_i, \delta_i, (z, t)) = \mathbf{b}_1(s_i)^\top \mathbf{k}(z, t, \delta_i), \quad Q_\theta(s_i, \delta_i, (z, t)) = \mathbf{b}_2(s_i)^\top \mathbf{k}(z, t, \delta_i)$$

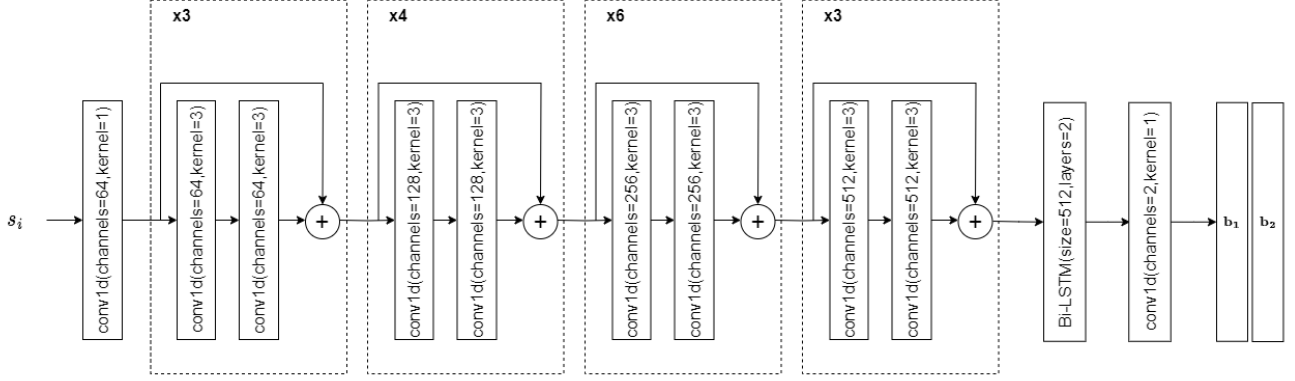


Figure 7: Structure of the branch net.

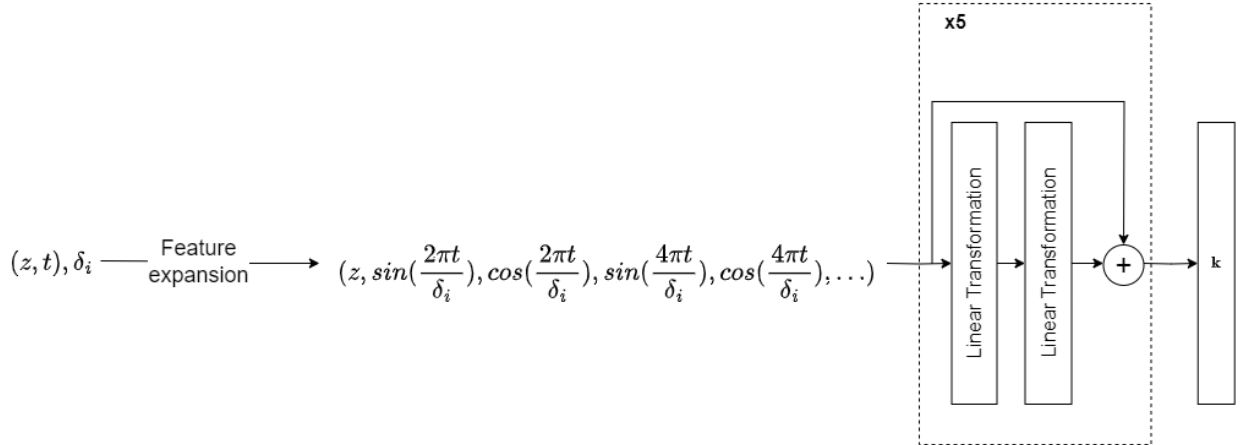


Figure 8: Structure of the trunk net.

where θ is the set of all trainable parameters in the BP-DeepONet.

The training loss function of our network is defined as

$$\mathcal{L}(P_\theta, Q_\theta) = \mathcal{R}_{pde}(P_\theta, Q_\theta) + \omega_1 \mathcal{R}_{b_1}(P_\theta) + \omega_2 \mathcal{R}_{b_2}(P_\theta, Q_\theta) \quad (9)$$

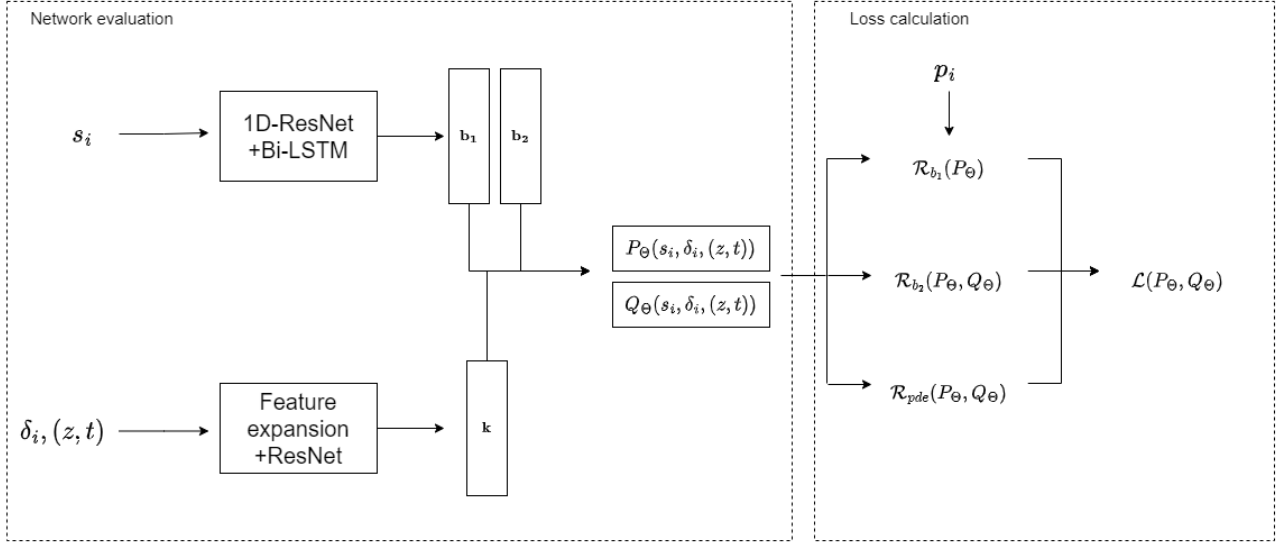


Figure 9: Structure of the BP-DeepONet

where

$$\mathcal{R}_{pde}(P_\theta, Q_\theta) = \frac{1}{N} \sum_{i=1}^N \left\| \left(\frac{\partial}{\partial t} \left(\frac{P_\theta}{Q_\theta} \right) + H(P_\theta, Q_\theta) \frac{\partial}{\partial z} \left(\frac{P_\theta}{Q_\theta} \right) - B(P_\theta, Q_\theta) \right) (s_i, \delta_i, \cdot) \right\|_{L^2([0, L] \times [0, T])}^2,$$

$$\mathcal{R}_{b_1}(P_\theta) = \frac{1}{N} \sum_{i=1}^N \|P_\theta(s_i, \delta_i, (L, \cdot)) - p_i\|_{L^2([0, T])}^2,$$

and

$$\mathcal{R}_{b_2}(P_\theta, Q_\theta) = \frac{1}{N} \sum_{i=1}^N \|W(P_\theta, Q_\theta)(s_i, \delta_i, (L, \cdot))\|_{L^2([0, T])}^2.$$

The complete structure of the BP-DeepONet is displayed in Figure 9. To evaluate the different residual terms $\mathcal{R}_{pde}$, $\mathcal{R}_{b_1}$, and $\mathcal{R}_{b_2}$, we need to approximate the L_2 norm with some discrete quadrature. For the PDE residual term $\mathcal{R}_{pde}$, we can simply approximate the i -th L_2 residual by

$$\left\| \left(\frac{\partial}{\partial t} \left(\frac{P_\theta}{Q_\theta} \right) + H(P_\theta, Q_\theta) \frac{\partial}{\partial z} \left(\frac{P_\theta}{Q_\theta} \right) - B(P_\theta, Q_\theta) \right) (s_i, \delta_i, \cdot) \right\|_{L^2([0, L] \times [0, T])}^2$$

$$\approx \sum_{j=1}^M \left| \left(\frac{\partial}{\partial t} \left(\frac{P_\theta}{Q_\theta} \right) + H(P_\theta, Q_\theta) \frac{\partial}{\partial z} \left(\frac{P_\theta}{Q_\theta} \right) - B(P_\theta, Q_\theta) \right) (s_i, \delta_i, (z_j, t_j)) \right|^2,$$

where $\{(z_j, t_j)\}_{j=1}^M$ is a set of quadrature points randomly sampled from the domain $[0, L] \times [0, T]$. The partial derivatives in the residual terms can be calculated as

$$\frac{\partial}{\partial x} P_\theta(s_i, \delta_i, (z_j, t_j)) = \mathbf{b}_1(s_i)^\top \frac{\partial}{\partial x} \mathbf{k}(\delta_i, (z_j, t_j)), \quad x = z \text{ or } t,$$

$$\frac{\partial}{\partial x} Q_\theta(s_i, \delta_i, (z_j, t_j)) = \mathbf{b}_2(s_i)^\top \frac{\partial}{\partial x} \mathbf{k}(\delta_i, (z_j, t_j)), \quad x = z \text{ or } t.$$

However, evaluating this approximation is very time-consuming because the total number of gradient evaluations in one training epoch is $O(MN)$. It is necessary to design a more efficient method to approximate the residuals. This can be achieved by utilizing the periodicity of our BP-DeepONet.

4.3. Efficient implementation of training BP-DeepONet

Given any periodic function $f(t)$ with period δ , we have the norm $\|\cdot\|_{L^2([0,T])}$ is equivalent to $\|\cdot\|_{L^2([0,\delta])}$:

$$\left\lfloor \frac{T}{\delta} \right\rfloor \|f\|_{L^2([0,\delta])}^2 \leq \|f\|_{L^2([0,T])}^2 \leq \left\lceil \frac{T}{\delta} \right\rceil \|f\|_{L^2([0,\delta])}^2,$$

where $\lfloor \cdot \rfloor$ and $\lceil \cdot \rceil$ denote the floor function and ceiling function, respectively. Then, for the i -th L_2 residual term in $\mathcal{R}_{pde}$, we may replace it with

$$\begin{aligned} & \left\| \left(\frac{\partial}{\partial t} \left(\frac{P_\theta}{Q_\theta} \right) + H(P_\theta, Q_\theta) \frac{\partial}{\partial z} \left(\frac{P_\theta}{Q_\theta} \right) - B(P_\theta, Q_\theta) \right) (s_i, \delta_i, \cdot) \right\|_{L^2([0,L] \times [0,\delta_i])}^2 \\ &= \int_0^{\delta_i} \int_0^L \left| \left(\frac{\partial}{\partial t} \left(\frac{P_\theta}{Q_\theta} \right) + H(P_\theta, Q_\theta) \frac{\partial}{\partial z} \left(\frac{P_\theta}{Q_\theta} \right) - B(P_\theta, Q_\theta) \right) (s_i, \delta_i, (z, t)) \right|^2 dz dt \\ &= \int_0^{\delta_i} \int_0^L \left| \left(\frac{\partial}{\partial t} \left(\frac{P_\theta}{Q_\theta} \right) + H(P_\theta, Q_\theta) \frac{\partial}{\partial z} \left(\frac{P_\theta}{Q_\theta} \right) - B(P_\theta, Q_\theta) \right) (s_i, 1, (z, t/\delta_i)) \right|^2 dz dt. \end{aligned}$$

After a change of variable $\hat{t} = t/\delta_i$, the above integration can be written as

$$\begin{aligned} & \int_0^1 \int_0^L \left| \left(\frac{\partial}{\partial \hat{t}} \left(\frac{P_\theta}{Q_\theta} \right) / \delta_i + H(P_\theta, Q_\theta) \frac{\partial}{\partial z} \left(\frac{P_\theta}{Q_\theta} \right) - B(P_\theta, Q_\theta) \right) (s_i, 1, (z, \hat{t})) \right|^2 dz d\hat{t} \\ & \approx \sum_{j=1}^M \left| \left(\frac{\partial}{\partial \hat{t}} \left(\frac{P_\theta}{Q_\theta} \right) / \delta_i + H(P_\theta, Q_\theta) \frac{\partial}{\partial z} \left(\frac{P_\theta}{Q_\theta} \right) - B(P_\theta, Q_\theta) \right) (s_i, 1, (z_j, \hat{t}_j)) \right|^2 \end{aligned}$$

where $\{(z_j, \hat{t}_j)\}_{j=1}^M$ is randomly sampled from the domain $[0, L] \times [0, 1]$. We approximate $\mathcal{R}_{pde}$ by

$$\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^M \left| \left(\frac{\partial}{\partial \hat{t}} \left(\frac{P_\theta}{Q_\theta} \right) / \delta_i + H(P_\theta, Q_\theta) \frac{\partial}{\partial z} \left(\frac{P_\theta}{Q_\theta} \right) - B(P_\theta, Q_\theta) \right) (s_i, 1, (z_j, \hat{t}_j)) \right|^2 \quad (10)$$

Then, the partial derivatives in (10) can be calculated as

$$\begin{aligned} \frac{\partial}{\partial x} P_\theta(s_i, \delta_i, (z_j, \hat{t}_j)) &= \mathbf{b}_1(s_i)^\top \frac{\partial}{\partial x} \mathbf{k}(1, (z_j, \hat{t}_j)), \quad x = z \text{ or } \hat{t}, \\ \frac{\partial}{\partial x} Q_\theta(s_i, 1, (z_j, \hat{t}_j)) &= \mathbf{b}_2(s_i)^\top \frac{\partial}{\partial x} \mathbf{k}(1, (z_j, \hat{t}_j)), \quad x = z \text{ or } \hat{t}. \end{aligned}$$

For different i , we do not need to recompute the gradient of $\mathbf{k}$, so the number of gradient evaluations in one training epoch is only $O(M)$. We can use the same idea to approximate $\mathcal{R}_{b2}$ by

$$\mathcal{R}_{b2}(P_\theta, Q_\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{j=1}^M \left| \left(Q_\theta(1 + R_1/R_2) + CR_1 \frac{\partial}{\partial \hat{t}} Q_\theta - P_\theta/R_2 - C \frac{\partial}{\partial \hat{t}} P_\theta \right) (s_i, 1, (L, \hat{t}_j)) \right|^2 \quad (11)$$

For $\mathcal{R}_{b1}$, we simply approximate it by

$$\mathcal{R}_{b1}(P_\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{j=1}^M |P_\theta(L, \hat{t}_j) - \bar{p}_i(\hat{t}_j \delta_i)|^2 \quad (12)$$

where $\{\hat{t}_j\}_{j=1}^M$ is randomly sampled from the domain $[0, 1]$ and $\bar{p}_i$ is the average of p_i over all periods contained in $[0, T]$. Finally, the entire empirical loss function for training the BP-DeepONet is the combination of three terms (10), (11), and (12):

$$\begin{aligned}\hat{\mathcal{L}}(P_\theta, Q_\theta) = & \frac{1}{N} \sum_{i=1}^N \sum_{j=1}^M \left| \left(\frac{\partial}{\partial \hat{t}} \left(\frac{P_\theta}{Q_\theta} \right) / \delta_i + H(P_\theta, Q_\theta) \frac{\partial}{\partial z} \left(\frac{P_\theta}{Q_\theta} \right) - B(P_\theta, Q_\theta) \right) (s_i, 1, (z_j, \hat{t}_j)) \right|^2 \\ & + \frac{\omega_1}{N} \sum_{i=1}^N \sum_{j=1}^M |P_\theta(s_i, 1, (L, \hat{t}_j)) - \bar{p}_i(\hat{t}_j \delta_i)|^2 \\ & + \frac{\omega_2}{N} \sum_{i=1}^N \sum_{j=1}^M \left| \left(Q_\theta(1 + R_1/R_2) + CR_1 \frac{\partial}{\partial \hat{t}} Q_\theta - P_\theta/R_2 - C \frac{\partial}{\partial \hat{t}} P_\theta \right) (s_i, 1, (L, \hat{t}_j)) \right|^2.\end{aligned}$$

4.4. Meta BP-DeepONet for sample-dependent hyper-parameters estimation

In the Navier-Stokes equation, many hyper-parameters depend on the physical properties of arterial systems. In the previous BP-DeepONet, we set all these hyper-parameters to be subject-independent, i.e., we use the same set of hyper-parameters for all samples. However, in real situations, these parameters differ from person to person, so it is unrealistic to set them all as constants. To handle this issue, we generalize the model (5) to be sample-dependent. For each training sample i from 1 to N , we denote the set of all hyper-parameters for the i -th sample as

$$\gamma_i := \{\beta_i, A_{0,i}, \rho_i, K_{r,i}, P_{ext,i}, R_{1,i}, R_{2,i}, C_i\},$$

and we assume the solutions P_i, Q_i for the i -th sample satisfy the following PDE:

$$\frac{\partial U_i}{\partial t} + H(U_i, \gamma_i) \frac{\partial U_i}{\partial z} = B(U_i, \gamma_i), \quad (z, t) \in [0, L] \times [0, T] \quad (13)$$

where

$$\begin{aligned}U_i &= \begin{pmatrix} P_i \\ Q_i \end{pmatrix}, \\ H(U_i, \gamma_i) &= \begin{pmatrix} 0 & \frac{\beta_i}{2a(P_i, \gamma_i)A_{0,i}} \\ \frac{a(P_i, \gamma_i)^2}{\rho_i} - \frac{2A_{0,i}Q_i^2}{\beta_i a^3(P_i, \gamma_i)} & \frac{2Q_i}{a(P_i, \gamma_i)^2} \end{pmatrix}, \\ B(U_i, \gamma_i) &= \begin{pmatrix} 0 \\ -K_{r,i} \frac{Q_i}{a(P_i, \gamma_i)^2} \end{pmatrix},\end{aligned}$$

and

$$a(P_i, \gamma_i) = \frac{A_{0,i}}{\beta_i} (P_i - P_{ext,i}) + \sqrt{A_{0,i}}.$$

The corresponding boundary and periodic conditions are:

$$\begin{aligned}P_i(L, t) &= p_i(t), \quad t \in [0, T], \\ W(P_i, Q_i, \gamma_i) &:= Q_i(L, t) \left(1 + \frac{R_{1,i}}{R_{2,i}} \right) + C_i R_{1,i} \frac{\partial Q_i}{\partial t}(L, t) - \frac{P_i(L, t)}{R_{2,i}} - C_i \frac{\partial P_i}{\partial t}(L, t) = 0, \quad t \in [0, T],\end{aligned} \quad (14)$$

and

$$U_i(z, t + \delta_i) = U_i(z, t), \quad z \in [0, L].$$

It is almost impossible to measure γ_i for each sample, so we adopt the idea from the field of meta-learning to estimate γ_i . We further define a hyper-network on top of the branch net. This network takes $\mathbf{b}_1$ and $\mathbf{b}_2$ as inputs and outputs the estimation of γ_i . This hyper-network is optimized together with the BP-DeepONet. We also impose extra constraints to regularize the output of the hyper-network. The first constraint is imposed as:

$$\int_0^T P_i(z, t) dt = \int_0^T p_i(t) dt, \quad \forall z \in [0, L], \quad (15)$$

which implies that the mean blood pressure does not change during the wave propagation. This phenomenon was found in clinical observations [54]. The second constraint is imposed as

$$R_{1,i} + R_{2,i} = \frac{\int_0^{\delta_i} P_i(L, t) dt}{\int_0^{\delta_i} Q_i(L, t) dt}. \quad (16)$$

This equation can be derived by integrating the equation (14) over one period $[0, \delta_i]$ and it is used for regularizing the estimation of $R_{1,i}$ and $R_{2,i}$. If we have other information about the hemodynamics, like the pulse wave velocity, we may introduce them as constraints to regularize the hyper-networks. Then, the training loss function of the meta BP-DeepONet is

$$\begin{aligned} \mathcal{L}_{meta}(P_\theta, Q_\theta, \gamma_\theta) = & \mathcal{R}_{pde}(P_\theta, Q_\theta, \gamma_\theta) \\ & + \omega_1 \mathcal{R}_{b_1}(P_\theta) + \omega_2 \mathcal{R}_{b_2}(P_\theta, Q_\theta, \gamma_\theta) + \omega_3 \mathcal{R}_{c_1}(P_\theta, Q_\theta) + \omega_4 \mathcal{R}_{c_2}(P_\theta, Q_\theta, \gamma_\theta) \end{aligned} \quad (17)$$

where Γ_θ is the set of estimated hyper-parameters given by the hyper-network:

$$\gamma_\theta(s_i) = \{\beta_\theta(s_i), A_{0,\theta}(s_i), \rho_\theta(s_i), K_{r,\theta}(s_i), P_{ext,\theta}(s_i), R_{1,\theta}(s_i), R_{2,\theta}(s_i), C_\theta(s_i)\}, \quad i = 1, \dots, N,$$

and the residual terms in (17) are defined as

$$\begin{aligned} \mathcal{R}_{pde}(P_\theta, Q_\theta, \gamma_\theta) &= \frac{1}{N} \sum_{i=1}^N \left\| \left(\frac{\partial}{\partial t} \left(\frac{P_\theta}{Q_\theta} \right) + H(P_\theta, Q_\theta, \gamma_\theta) \frac{\partial}{\partial z} \left(\frac{P_\theta}{Q_\theta} \right) - B(P_\theta, Q_\theta, \gamma_\theta) \right) (s_i, \delta_i, \cdot) \right\|_{L^2([0,L] \times [0,T])}^2, \\ \mathcal{R}_{b_2}(P_\theta, Q_\theta, \gamma_\theta) &= \frac{1}{N} \sum_{i=1}^N \|W(P_\theta, Q_\theta, \gamma_\theta)(s_i, \delta_i, (L, \cdot))\|_{L^2([0,T])}^2, \\ \mathcal{R}_{c_1}(P_\theta, Q_\theta) &= \frac{1}{N} \sum_{i=1}^N \left\| \int_0^T P_\theta(s_i, \delta_i, (\cdot, t)) dt - \int_0^T p_i(t) dt \right\|_{L^2([0,L])}^2, \\ \mathcal{R}_{c_2}(P_\theta, Q_\theta, \gamma_\theta) &= \frac{1}{N} \sum_{i=1}^N \left(R_{1,\theta}(s_i) + R_{2,\theta}(s_i) - \frac{\int_0^{\delta_i} P_\theta(s_i, \delta_i, (L, t)) dt}{\int_0^{\delta_i} Q_\theta(s_i, \delta_i, (L, t)) dt} \right)^2, \end{aligned}$$

and $\mathcal{R}_{b_1}(P_\theta)$ is defined the same as (9). To approximate these residual terms, we use the same change of variable technique as section 4.2:

$$\begin{aligned} \mathcal{R}_{pde}(P_\theta, Q_\theta, \gamma_\theta) &\approx \frac{1}{N} \sum_{i=1}^N \sum_{j=1}^M \left| \left(\frac{\partial}{\partial t} \left(\frac{P_\theta}{Q_\theta} \right) + H(P_\theta, Q_\theta, \gamma_\theta) \frac{\partial}{\partial z} \left(\frac{P_\theta}{Q_\theta} \right) - B(P_\theta, Q_\theta, \gamma_\theta) \right) (s_i, 1, (z_j, \hat{t}_j)) \right|^2, \\ \mathcal{R}_{b_2}(P_\theta, Q_\theta, \gamma_\theta) &\approx \frac{1}{N} \sum_{i=1}^N \sum_{j=1}^M \left| \left(Q_\theta(1 + R_{1,\theta}/R_{2,\theta}) + C_\theta R_{1,\theta} \frac{\partial}{\partial \hat{t}} Q_\theta - P_\theta/R_{2,\theta} - C_\theta \frac{\partial}{\partial \hat{t}} P_\theta \right) (s_i, 1, (L, \hat{t}_j)) \right|^2, \\ \mathcal{R}_{c_1}(P_\theta, Q_\theta) &\approx \frac{1}{N} \sum_{i=1}^N \sum_{j_1=1}^{M'} \left| \sum_{j_2=1}^M (P_\theta(s_i, 1, (\hat{z}_{j_1}, \hat{t}_{j_2})) - p_i(\hat{t}_{j_2})) \right|^2, \\ \mathcal{R}_{c_2}(P_\theta, Q_\theta, \gamma_\theta) &\approx \frac{1}{N} \sum_{i=1}^N \left| R_{1,\theta}(s_i) + R_{2,\theta}(s_i) - \frac{\sum_{j=1}^M P_\theta(s_i, \delta_i, (L, \hat{t}_j))}{\sum_{j=1}^M Q_\theta(s_i, \delta_i, (L, \hat{t}_j))} \right|^2, \end{aligned}$$

where $\{(z_j, \hat{t}_j)\}_{j=1}^M$ is a set of quadrature points sampled from $[0, L] \times [0, 1]$ and $\{\hat{z}_j\}_{j=1}^{M'}$ is a set of quadrature points sampled from $[0, L]$, and $\mathcal{R}_{b_1}(P_\theta)$ is approximated in the same way as (12). Then, the empirical training loss for the

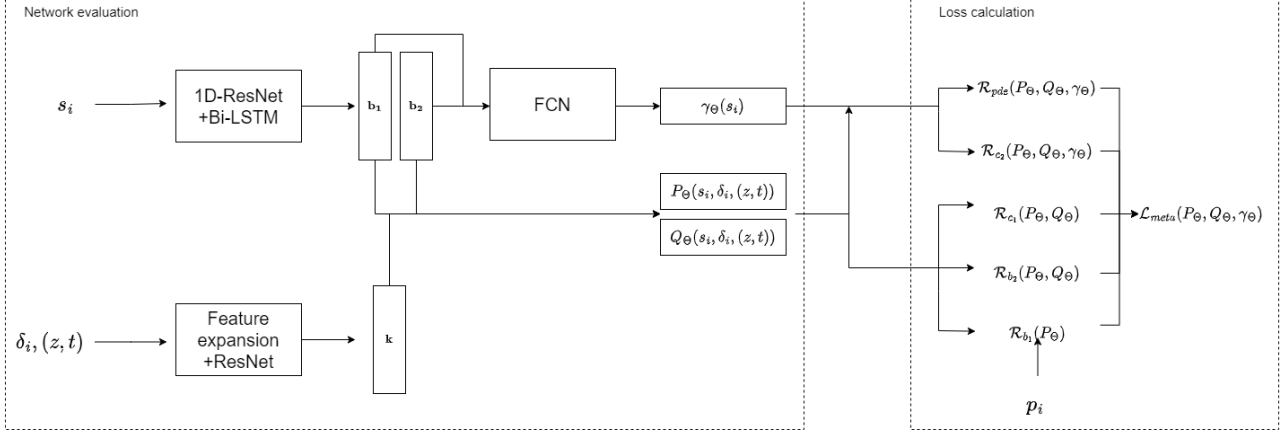


Figure 10: Structures of the meta BP-DeepONet

meta BP-DeepONet is given as

$$\begin{aligned}
\hat{\mathcal{L}}_{meta}(P_\theta, Q_\theta, \gamma_\theta) = & \frac{1}{N} \sum_{i=1}^N \sum_{j=1}^M \left| \left(\frac{\partial}{\partial t} \begin{pmatrix} P_\theta \\ Q_\theta \end{pmatrix} + H(P_\theta, Q_\theta, \gamma_\theta) \frac{\partial}{\partial z} \begin{pmatrix} P_\theta \\ Q_\theta \end{pmatrix} - B(P_\theta, Q_\theta, \gamma_\theta) \right) (s_i, 1, (z_j, \hat{t}_j)) \right|^2 \\
& + \frac{\omega_1}{N} \sum_{i=1}^N \sum_{j=1}^M |P_\theta(s_i, 1, (L, \hat{t}_j)) - \bar{p}_i(\hat{t}_j \delta_i)|^2 \\
& + \frac{\omega_2}{N} \sum_{i=1}^N \sum_{j=1}^M \left| \left(Q_\theta(1 + R_{1,\theta}/R_{2,\theta}) + C_\theta R_{1,\theta} \frac{\partial}{\partial \hat{t}} Q_\theta - P_\theta/R_{2,\theta} - C_\theta \frac{\partial}{\partial \hat{t}} P_\theta \right) (s_i, 1, (L, \hat{t}_j)) \right|^2 \\
& + \frac{\omega_3}{N} \sum_{i=1}^N \sum_{j_1=1}^{M'} \left| \sum_{j_2=1}^M (P_\theta(s_i, 1, (\hat{z}_{j_1}, \hat{t}_{j_2})) - p_i(\hat{t}_{j_2})) \right|^2 \\
& + \frac{\omega_4}{N} \sum_{i=1}^N \left| R_{1,\theta}(s_i) + R_{2,\theta}(s_i) - \frac{\sum_{j=1}^M P_\theta(s_i, \delta_i, (L, \hat{t}_j))}{\sum_{j=1}^M Q_\theta(s_i, \delta_i, (L, \hat{t}_j))} \right|^2.
\end{aligned}$$

The overall structures of the meta BP-DeepONet are given in Figure 10 where the hyper-network is implemented as a simple FCN.

Remark. During the training of the meta BP-DeepONet, we only require the measurement of blood pressure at $z = L$. Then the whole solution, all boundary conditions, initial conditions, and hyper-parameters can be automatically learned from this measurement. No extra measurements and parameters are needed.

5. Numerical experiments of the PINN on simulated data

In this section, we would like to validate the proposed physics-informed training method by solving one PDE instance. We first generate a solution to the Navier-Stokes equation (4) using a MacCormack scheme with a prescribed initial condition and inlet boundary condition. Then, we solve this equation with the boundary conditions (7) and (6) and the time-periodic condition (8) using a simple PINN. We can see in the numerical results that the PINN solution is very close to the simulated solution.

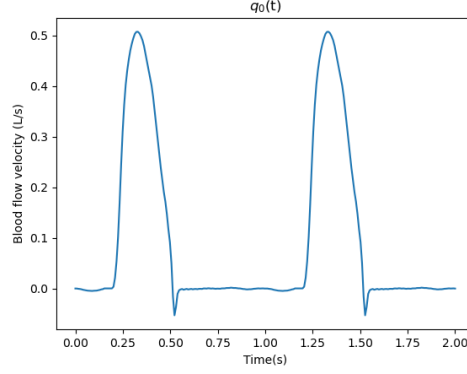


Figure 11: The inflow boundary condition

We consider the 1D Navier-stokes equation (1) and (2) with the initial condition:

$$A(x, 0) = A_0, \quad Q(x, 0) = 0,$$

the inflow boundary condition:

$$Q(0, t) = q_0(t),$$

and the three-element Windkessel boundary condition

$$Q(L, t)(1 + \frac{R_1}{R_2}) + CR_1 \frac{\partial Q}{\partial t}(L, t) = \frac{P(L, t)}{R_2} + C \frac{\partial P}{\partial t}(L, t).$$

Such an initial boundary value problem can be efficiently solved by many classical methods like the MacCormack scheme [26, 27], Taylor-Galerkin scheme [28, 29], MUSCL [30, 31], and the local discontinuous Galerkin scheme [32–34]. We use a numerical example that is similar to the baseline aorta case provided in [21], where the hyper-parameters are chosen as $L = 25\text{cm}$, $A_0 = 4.52\text{cm}^2$, $\beta = 1134.37\text{kg/s}^2$, $\rho = 1060\text{kg/m}^3$, $P_{ext} = 9\text{kPa}$, $R_1 = 1.17 \times 10^7\text{Pa s m}^{-3}$, $R_2 = 1.12 \times 10^8\text{Pa s m}^{-3}$, $C = 1.01 \times 10^{-8}\text{m}^3\text{Pa}^{-1}$, $v = 0.01 \times 10^6\text{m}^2/\text{s}^2$, and the inflow boundary conditions $q_0(t)$ is given in Figure 11. Then, we solve this problem using the MacCormack scheme.

After having a referenced numerical solution $\hat{A}$ and $\hat{Q}$, we define a residual network $N_\theta(z, t) = (P_\theta(z, t), Q_\theta(z, t))$ shown in Figure 4 (b) with five residual layers. The time-periodic condition is incorporated into the network using the same feature expansion technique described before. We train this network by minimizing the following loss function:

$$\begin{aligned} L_{PINN}(\theta) := & \left\| \frac{\partial A_\theta}{\partial t} + \frac{\partial Q_\theta}{\partial z} \right\|_{L^2([0, L] \times [0, T])}^2 + \left\| \frac{\partial Q_\theta}{\partial t} + \frac{\partial}{\partial z} \left(\frac{Q_\theta^2}{A_\theta} \right) + \frac{A_\theta}{\rho} \frac{\partial P_\theta}{\partial z} + K_r \frac{Q_\theta}{A_\theta} \right\|_{L^2([0, L] \times [0, T])}^2 \\ & + 100 \left(\|P_\theta(L, \cdot) - \hat{P}(L, \cdot)\|_{L^2([0, T])}^2 + \left\| Q_\theta(L, \cdot) \left(1 + \frac{R_1}{R_2} \right) + CR_1 \frac{\partial Q_\theta}{\partial t}(L, \cdot) - \frac{P_\theta(L, \cdot)}{R_2} - C \frac{\partial P_\theta}{\partial t}(L, \cdot) \right\|_{L^2([0, T])}^2 \right) \end{aligned}$$

where $P_\theta(z, t) = P_{ext} + \frac{\beta}{A_0}(\sqrt{A_\theta(z, t)} - \sqrt{A_0})$. We use a standard Adam optimizer with a learning rate of $1e-4$, and the total number of training epochs is 50,000. We randomly sample 1,000 quadrature points in the domain in each epoch to approximate the L^2 integration. We can see the comparison of the PINN solution and the numerical solution by the MacCormack scheme in Figure 12. The PINN solution solved using the loss function L_{PINN} is very close to the solution of the MacCormack scheme. We also calculate the relative error:

$$\left\| \frac{\hat{P} - P_\theta}{\hat{P}} \right\|_1 \approx 0.30\%$$

and

$$\frac{\|\hat{Q} - Q_\theta\|_1}{\|\hat{Q}\|_\infty} \approx 0.67\%.$$

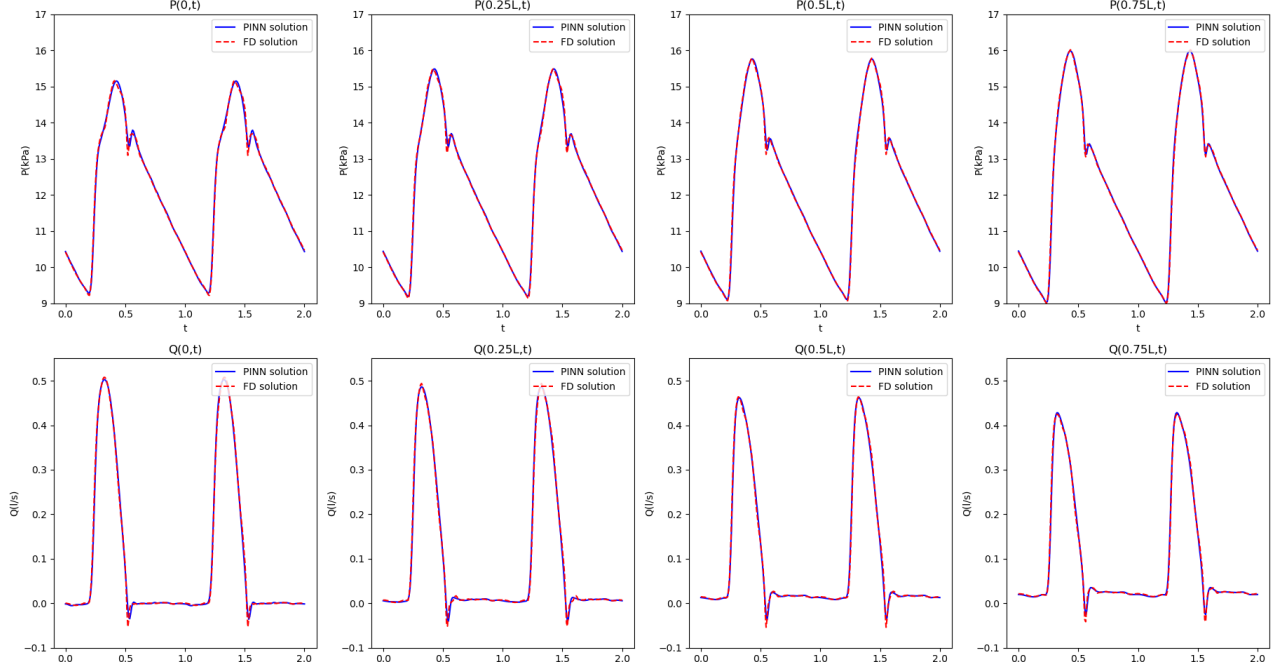


Figure 12: Comparison of the PINN solution and the finite difference solution (MacCormack).

These two metrics are also used in [21] to measure the error of their numerical solutions for the Navier-Stokes equation. From Figure 12, we can observe that the error of the PINN solution is larger near the dicrotic notch where the solution is near singular. This case is very common when using the standard PINN method to solve PDEs, and it can be improved by using some adaptive sampling scheme [55]. We may consider this direction in our future work.

6. Numerical experiments of the PI-DeepONet methods on clinical data

6.1. Dataset description

We conducted our experiments on the MIMIC dataset [56], the largest and most commonly used dataset for cuffless blood pressure estimation. We use the pre-processed version of the MIMIC data from [57], which contains recordings of 12,000 subjects after simple pre-processing. Each recording consists of continuous ECG, PPG, and ABP waveform data. The length of each recording is within 10 minutes. All these data are collected from ICU patients, and the invasive method collects the ABP waveform data. Furthermore, we also apply some extra pre-processing steps to these data:

1. Use a bandpass filter to remove noise from PPG and ABP signals.
2. Align the PPG and ABP signals.
3. Randomly select 75% of subjects for training and 25% for testing.
4. 20% of the training set is selected as the validation set.
5. The first 15% of data in each testing subject is used for calibration.
6. Split each recording into many 4-second segments without overlapping.
7. Remove those segments that are highly non-periodic.
8. Remove subjects whose number of valid segments is less than 10.
9. Apply some peak detection algorithms to detect all peaks in each PPG segment, then calculate the average length of intervals between two consecutive peaks.

After pre-processing, the total number of training and testing samples is about 70,000. Step 5 is also called individual fine-tuning [58], and it is a necessary process to calibrate and improve the blood pressure measurement. The segments for calibration will not be used when calculating the testing error. One segment of the recording is shown in Figure 13.

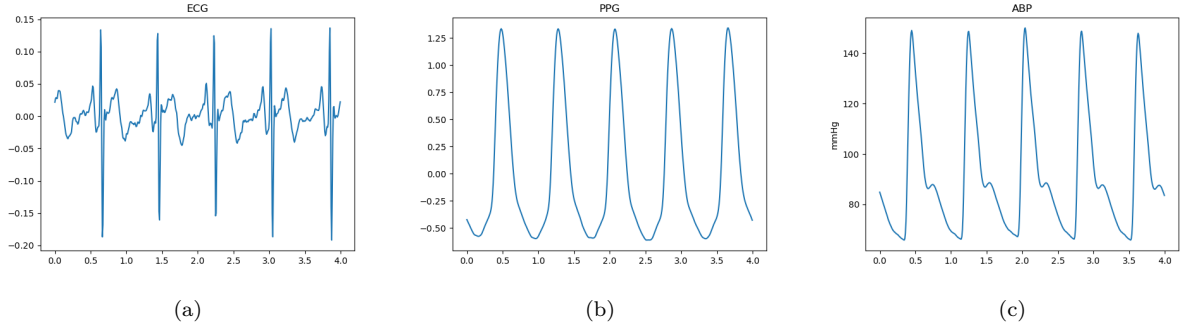


Figure 13: One segment of ECG, PPG, and ABP signals.

6.2. Training procedures

For the training of the BP-DeepONet and the Meta BP-DeepONet, We apply the standard Adam optimizer with a scheduled learning rate to train the network. The initial learning rate is set to be $1e-4$, and decays by a factor of 0.9 every 5 epochs. After 200 epochs, we restart the learning rate with $1e-4$ and continue to train with the same learning rate schedule. After several rounds of learning rate cycles, we choose the best model based on the performance evaluated on a validation set.

When training the BP-DeepONet, we need first to determine all the hyper-parameters in the NS model, so we set hyper-parameters based on the reference values of the radial artery described in [59] and estimate the Windkessel parameters using the method provided in [21]: $A_0 = 8 \times 10^{-6} m^2$, $\beta = 529 kg/s^2$, $L = 0.25m$, $K_r = 8 * \pi * 0.01 \approx 0.251$, $\rho = 1.06 kg/m^3$, $P_{ext} = 6.67 kPa$, $R_1 = 12.37 Pa \cdot s/m^3$, $R_2 = 137.5 Pa \cdot s/m^3$, $C = 0.01 m^3/Pa$. In the Meta BP-DeepONet, we also use these values as referenced values for the predicted hyper-parameters. We restrict the deviation of the predicted hyper-parameters from the referenced values to be less than or equal to 20 % so that the estimation of the hyper-parameters will always be reasonable.

During the training of both BP-DeepONet and meta BP-DeepONet, we use a batch size of 100. All the training is done on a GPU server with 4 NVIDIA RTX A6000 GPUs.

6.3. Numerical results

After finishing the training of DeepONet, we can obtain the prediction of the BP waveform at any given location z by evaluating the DeepONet at $(z, t_0), (z, t_1), \dots, (z, t_{N_h})$, where $\{t_j\}_{j=0}^{N_h}$ is a uniform discretization of the domain $[0, T]$.

We first evaluate the waveform prediction at the outlet side $z = L$. Since we have the measured BP wave at $z = L$ by the invasive method, we can compute the prediction error of our method at this location. We evaluate the error using several metrics: waveform MAE, SBP MAE, DBP MAE, and MBP MAE, where MAE is short for mean absolute error. Suppose $P_i = (P_\theta(s_i, \delta_i, (L, t_0)), P_\theta(s_i, \delta_i, (L, t_1)), \dots, P_\theta(s_i, \delta_i, (L, t_{N_h}))) \in \mathbb{R}^{N_h}$ is the predicted waveform and $p_i \in \mathbb{R}^{N_h}$ is the ground true BP waveform. Then, the four metrics are defined as follows:

- Waveform MAE: $\frac{1}{N} \frac{1}{N_h} \sum_{i=1}^N |P_i - p_i|_1$,
- SBP MAE: $\frac{1}{N} \sum_{i=1}^N |\max(P_i) - \max(p_i)|$,
- DBP MAE: $\frac{1}{N} \sum_{i=1}^N |\min(P_i) - \min(p_i)|$,
- MBP MAE: $\frac{1}{N} \sum_{i=1}^N |\text{mean}(P_i) - \text{mean}(p_i)|$,

where $|\cdot|_1$ denotes the standard vector l_1 norm. In Table 1, we listed the error metrics for both BP-DeepONet and meta BP-DeepONet. We also compared our results with the IEEE standard for wearable, cuffless blood pressure measuring devices [60]. The suggested grading is also shown in Table 1. We can see the meta BP-DeepONet achieves better accuracy on the MAE of SBP, DBP, and MBP, while the BP-DeepONet achieves better accuracy on the waveform MAE. According to the IEEE standard, the meta BP-DeepONet is graded as A-level in all metrics, while the BP-DeepONet is graded as A-level except on the SBP measurement.

	Methods	Waveform	SBP	DBP	MBP
MAE (mmHg)	BP-DeepONet	4.406	5.057	2.581	2.881
	Meta BP-DeepONet	4.555	4.934	2.574	2.871
Grading	BP-DeepONet	A	B	A	A
	Meta BP-DeepONet	A	A	A	A

Table 1: The prediction error of two proposed methods and their grading according to the IEEE standard.

Since our methods can predict the blood pressure $P(z, t)$ and blood flow rate $Q(z, t)$ in the whole domain $(z, t) \in [0, L] \times [0, T]$, we also want to evaluate the errors in the interior of this domain. We choose two different metrics to evaluate the error in $(z, t) \in [0, L] \times [0, T]$. The first metric is the residual errors $\mathcal{R}_{pde}$. The residual error is usually used to estimate the total error when the true solution is unknown. For the BP-DeepONet, the residual error is defined as:

$$RE_1(P_\theta, Q_\theta) = \frac{1}{N} \sum_{i=1}^N \left\| \left(\frac{\partial A_\theta}{\partial t} + \frac{\partial Q_\theta}{\partial z} \right) (s_i, \delta_i, (\cdot, \cdot)) \right\|_{L^2([0, L] \times [0, T])}$$

$$RE_2(P_\theta, Q_\theta) = \frac{1}{N} \sum_{i=1}^N \left\| \left(\frac{\partial Q_\theta}{\partial t} + \frac{\partial}{\partial z} \left(\frac{Q_\theta^2}{A_\theta} \right) + \frac{A_\theta}{\rho} \frac{\partial P_\theta}{\partial z} + K_r \frac{Q_\theta}{A_\theta} \right) (s_i, \delta_i, (\cdot, \cdot)) \right\|_{L^2([0, L] \times [0, T])}$$

where $A_\theta = \left(\frac{A_0}{\beta} (P_\theta - P_{ext}) + \sqrt{A_0} \right)^2$. For the meta BP-DeepONet, the residual error is defined as:

$$RE_1(P_\theta, Q_\theta, \gamma_\theta) = \frac{1}{N} \sum_{i=1}^N \left\| \left(\frac{\partial A_{\theta,i}}{\partial t} + \frac{\partial Q_\theta}{\partial z} \right) (s_i, \delta_i, (\cdot, \cdot)) \right\|_{L^2([0, L] \times [0, T])}$$

$$RE_2(P_\theta, Q_\theta, \gamma_\theta) = \frac{1}{N} \sum_{i=1}^N \left\| \left(\frac{\partial Q_\theta}{\partial t} + \frac{\partial}{\partial z} \left(\frac{Q_\theta^2}{A_{\theta,i}} \right) + \frac{A_{\theta,i}}{\rho_i} \frac{\partial P_\theta}{\partial z} + K_{r_i} \frac{Q_\theta}{A_{\theta,i}} \right) (s_i, \delta_i, (\cdot, \cdot)) \right\|_{L^2([0, L] \times [0, T])}$$

where $A_{\theta,i} = \left(\frac{A_{0,i}}{\beta} (P_\theta - P_{ext,i}) + \sqrt{A_{0,i}} \right)^2$. In [43], the authors show that the error of solutions to the Navier-Stokes by neural networks can be bounded by the residual errors. The residual errors of the two methods are shown in Table 2.

	$RE_1(\sqrt{L})$	$RE_2(\sqrt{dm * L/s})$
BP-DeepONet	$1.11 \times 1e-4$	$2.65 \times 1e-2$
Meta BP-DeepONet	$0.22 \times 1e-4$	$1.14 \times 1e-2$

Table 2: The residual error of two proposed methods.

The second metric we used to evaluate the error is the relative error compared to a fine-tuned BP-DeepONet (or meta BP-DeepONet) on each test sample. Though we do not know the ground true solution, we have shown in Section 5 that a well-trained PINN solution is reasonably accurate compared to the simulated true solution. Therefore, we can fine-tune the network on each test sample, i.e., minimize the PDE residual loss of this particular sample only, and use this fine-tuned network as an approximation to the true solution. During the fine-tuning stage, we fix the parameters in the branch net and only train the parameters in the trunk net so that predictions of hyper-parameters will not change. The error metric is defined as:

$$\sum_{i=1}^N \frac{\left\| \left(\hat{P}_{tuned} - P_\theta \right) (s_i, \delta_i, (\cdot, \cdot)) \right\|_{L^1([0, L] \times [0, T])}}{\left\| \hat{P}_{tuned}(s_i, \delta_i, (\cdot, \cdot)) \right\|_{L^1([0, L] \times [0, T])}}$$

and

$$\sum_{i=1}^N \frac{\left\| \left(\hat{Q}_{tuned} - Q_\theta \right) (s_i, \delta_i, (\cdot, \cdot)) \right\|_{L^1([0, L] \times [0, T])}}{\left\| \hat{Q}_{tuned}(s_i, \delta_i, (\cdot, \cdot)) \right\|_{L^\infty([0, L] \times [0, T])}}$$

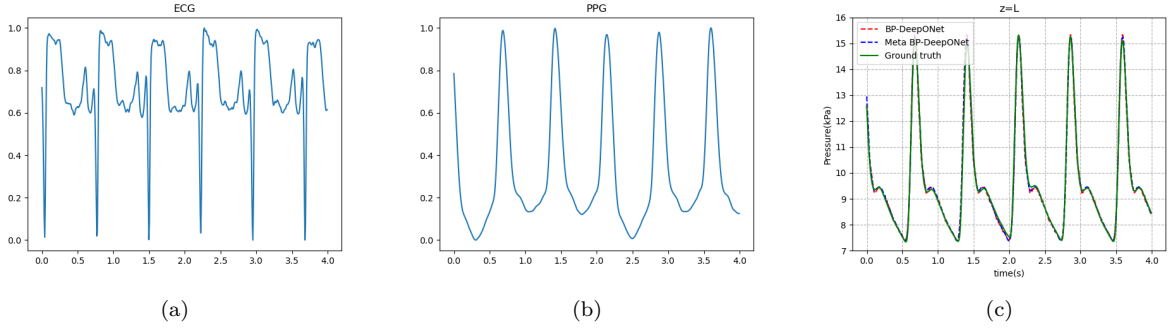


Figure 14: (a) and (b) show the input physiological signals. (c) shows the prediction and ground truth data at $z = L$.

where $\hat{P}_{tuned}$ and $\hat{Q}_{tuned}$ denote the solution after fine-tuning. The relative errors are given in Table 3. We can see the BP-DeepONet can predict the PDE solution better than the Meta BP-DeepONet. This result is not surprising because the BP-DeepONet assumes all samples satisfy the same PDE, which greatly simplifies the problem.

	error of P_θ	error of Q_θ
BP-DeepONet	$4.10 \times 1e-2$	$5.35 \times 1e-2$
Meta BP-DeepONet	$4.18 \times 1e-2$	$1.03 \times 1e-1$

Table 3: The relative error of two proposed methods.

We also visualize the predicted waveforms for one test sample in Figure 14 and Figure 15. We can observe that the predictions of both the BP-DeepONet and the meta BP-DeepONet are very close to the ground truth data at the outlet side $z = L$ (Figure 14 (c)). We can also observe the pulse pressure amplification with the difference between the maximum and minimum of the waveforms increases when z increases for both methods in Figure 15. Because the Meta BP-DeepONet uses different hyper-parameters for each sample, its prediction differs from BP-DeepONet in $z \in (0, 1)$, especially at $z = 0$.

7. Conclusions and Future Works

In this work, we propose two methods based on the physics-informed DeepONet to predict the blood pressure waveform: the BP-DeepONet and the meta BP-DeepONet. We incorporate the Navier-Stokes equation with time-periodic conditions and Windkessel boundary conditions into the physics-informed training procedure and provide efficient implementation algorithms. The proposed methods are the first ones that can predict waveforms at different locations. During our numerical experiments, the proposed methods can predict blood pressure waveforms at reasonable accuracy and preserve some physical properties consistent with the clinical observations.

This work only considers very standard DNN models for trunk and branch nets. We may try implementing different DNN architectures in the future, like the transformer, which is very powerful in processing time-series data. Besides, the inputs to the DeepONet are just the pre-processed physiological signals. We may also try including some morphological features in the inputs, effectively improving the prediction accuracy.

We know the arterial system of humans is very complicated. As the first work in this direction, we only consider a simple cylindrical segment of an artery. Still, we may generalize our methods to more complex geometry afterward—for example, a simple arterial tree with bifurcations.

In the future, we will also consider generating a simulated dataset to validate better the accuracy of both BP-DeepONet and meta BP-DeepONet methods.

Acknowledgement

The work of Lingfeng Li is supported by the InnoHK project at Hong Kong Centre for Cerebro-cardiovascular Health Engineering (COCHE). The work of Xue-Cheng Tai is partially supported by HKRGC-NSFC Grant N-CityU214/19,

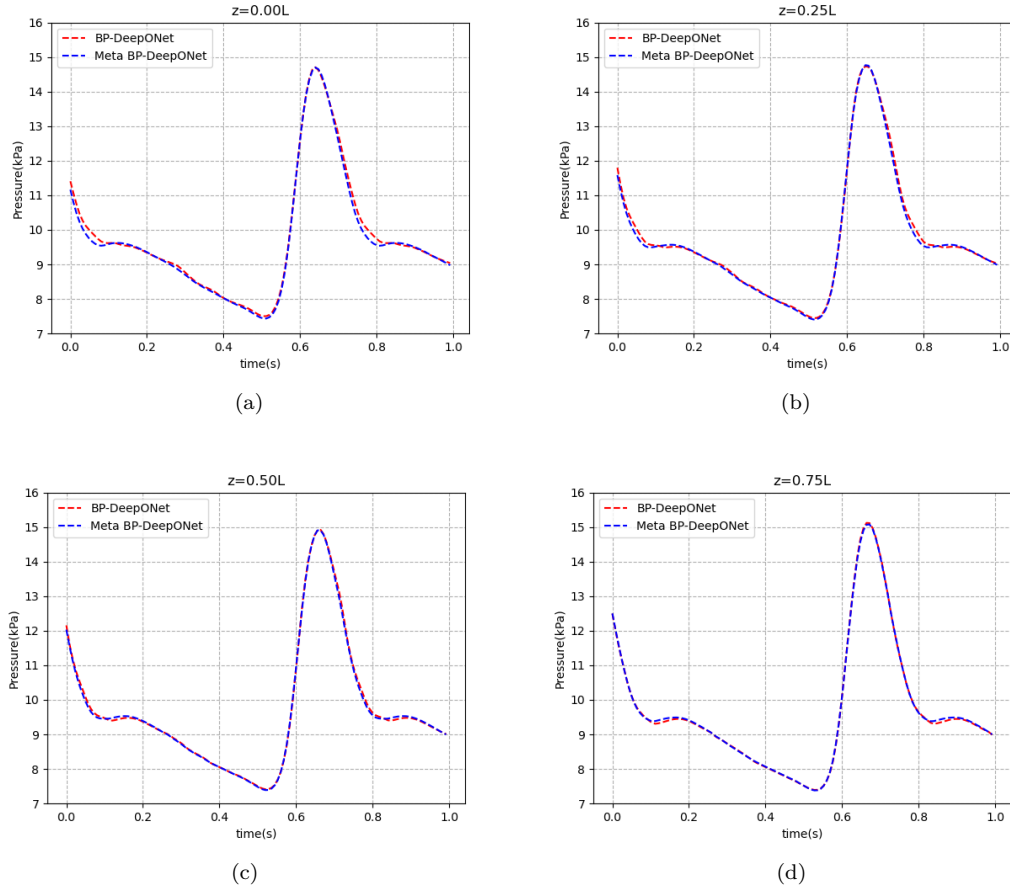


Figure 15: Predictions of ABP waveforms at different locations z .

HKRGC CRF. Grant C1013-21GF and NORCE Kompetanseoppbygging program. The work of Raymond Chan is partially supported by HKRGC GRF grants CityU1101120, CityU11309922, CRF grant C1013-21GF and HKITF MHKJFS Grant MHP/054/22. We want to thank Prof Yuan-Ting Zhang and Dr. Alireza Keramat for the helpful discussion during this project.

References

- [1] World Health Organization, Cardiovascular diseases (cvds), [https://www.who.int/news-room/fact-sheets/detail/cardiovascular-diseases-\(cvds\)](https://www.who.int/news-room/fact-sheets/detail/cardiovascular-diseases-(cvds)).
- [2] F. D. Fuchs, P. K. Whelton, High blood pressure and cardiovascular disease, *Hypertension* 75 (2) (2020) 285–292.
- [3] B. Matthieu, N.-G. Karine, C. Vincent, V. Alain, C. J. François, R. Philippe, S. François, Cardiac output measurement in patients undergoing liver transplantation: pulmonary artery catheter versus uncalibrated arterial pressure waveform analysis, *Anesthesia & Analgesia* 106 (5) (2008) 1480–1486.
- [4] R. H. Thiele, M. E. Durieux, Arterial waveform analysis for the anesthesiologist: past, present, and future concepts, *Anesthesia & Analgesia* 113 (4) (2011) 766–776.
- [5] Y. Li, H. Gu, H. Fok, J. Alastruey, P. Chowieńczyk, Forward and backward pressure waveform morphology in hypertension, *Hypertension* 69 (2) (2017) 375–381.

- [6] D. A. Hullender, O. R. Brown, Simulations of blood pressure and identification of atrial fibrillation and arterial stiffness using an extended kalman filter with oscillometric pulsation measurements, *Computer Methods and Programs in Biomedicine* 198 (2021) 105768.
- [7] P. Su, X.-R. Ding, Y.-T. Zhang, J. Liu, F. Miao, N. Zhao, Long-term blood pressure prediction with deep recurrent neural networks, in: 2018 IEEE EMBS International Conference on Biomedical & Health Informatics (BHI), IEEE, 2018, pp. 323–328.
- [8] H. Eom, D. Lee, S. Han, Y. S. Hariyani, Y. Lim, I. Sohn, K. Park, C. Park, End-to-end deep learning architecture for continuous blood pressure estimation using attention mechanism, *Sensors* 20 (8) (2020) 2338.
- [9] N. Ibtehaz, S. Mahmud, M. E. Chowdhury, A. Khandakar, M. Salman Khan, M. A. Ayari, A. M. Tahir, M. S. Rahman, Ppg2abp: Translating photoplethysmogram (ppg) signals to arterial blood pressure (abp) waveforms, *Bioengineering* 9 (11) (2022) 692.
- [10] C. Ma, P. Zhang, F. Song, Y. Sun, G. Fan, T. Zhang, Y. Feng, G. Zhang, Kd-informer: Cuff-less continuous blood pressure waveform estimation approach based on single photoplethysmography, *IEEE Journal of Biomedical and Health Informatics* (2022).
- [11] K. Lakhal, V. Robert-Edan, Invasive monitoring of blood pressure: a radiant future for brachial artery as an alternative to radial artery catheterisation?, *Journal of Thoracic Disease* 9 (12) (2017) 4812.
- [12] S.-P. E, Continuous non-invasive arterial pressure shows high accuracy in comparison to invasive intra-arterial blood pressure measurement, 2008.
URL <https://api.semanticscholar.org/CorpusID:30046018>
- [13] M. K. Armstrong, M. G. Schultz, D. S. Picone, J. A. Black, N. Dwyer, P. Roberts-Thomson, J. E. Sharman, Brachial and radial systolic blood pressure are not the same: evidence to support the popeye phenomenon, *Hypertension* 73 (5) (2019) 1036–1041.
- [14] J. Kyriazis, J. Glotsos, L. Bilirakis, N. Smirnioudis, The (dp/dt) max derived from arterial pulse waveforms: prospective applications in the haemodialysis setting, *Nephrology Dialysis Transplantation* 16 (5) (2001) 1087–1088.
- [15] J. Sharman, A. Qasem, L. Hanekom, D. Gill, R. Lim, T. Marwick, Radial pressure waveform dp/dt max is a poor indicator of left ventricular systolic function, *European Journal of Clinical Investigation* 37 (4) (2007) 276–281.
- [16] S. Wang, H. Wang, P. Perdikaris, Learning the solution operator of parametric partial differential equations with physics-informed deepnets, *Science Advances* 7 (40) (2021) eabi8605.
- [17] L. Lu, P. Jin, G. Pang, Z. Zhang, G. E. Karniadakis, Learning nonlinear operators via deepnet based on the universal approximation theorem of operators, *Nature Machine Intelligence* 3 (3) (2021) 218–229.
- [18] M. Raissi, P. Perdikaris, G. E. Karniadakis, Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, *Journal of Computational physics* 378 (2019) 686–707.
- [19] L. Formaggia, D. Lamponi, A. Quarteroni, One-dimensional models for blood flow in arteries, *Journal of Engineering Mathematics* 47 (2003) 251–276.
- [20] A. Quarteroni, L. Formaggia, Mathematical modelling and numerical simulation of the cardiovascular system, *Handbook of Numerical Analysis* 12 (2004) 3–127.
- [21] N. Xiao, J. Alastruey, C. Alberto Figueroa, A systematic comparison between 1-d and 3-d hemodynamics in compliant arterial models, *International Journal for Numerical Methods in Biomedical Engineering* 30 (2) (2014) 204–231.
- [22] J. Alastruey, A. W. Khir, K. S. Matthys, P. Segers, S. J. Sherwin, P. R. Verdonck, K. H. Parker, J. Peiró, Pulse wave propagation in a model human arterial network: assessment of 1-d visco-elastic simulations against in vitro measurements, *Journal of Biomechanics* 44 (12) (2011) 2250–2258.

- [23] M. Saito, Y. Ikenaga, M. Matsukawa, Y. Watanabe, T. Asada, P.-Y. Lagree, One-dimensional model for propagation of a pressure wave in a model of the human arterial network: Comparison of theoretical and experimental results, *Journal of Biomechanical Engineering* 133 (12) (2011).
- [24] M. S. Olufsen, C. S. Peskin, W. Y. Kim, E. M. Pedersen, A. Nadim, J. Larsen, Numerical simulation and experimental validation of blood flow in arteries with structured-tree outflow conditions, *Annals of Biomedical Engineering* 28 (2000) 1281–1299.
- [25] B. N. Steele, J. Wan, J. P. Ku, T. J. Hughes, C. A. Taylor, In vivo validation of a one-dimensional finite-element method for predicting blood flow in cardiovascular bypass grafts, *IEEE Transactions on Biomedical Engineering* 50 (6) (2003) 649–656.
- [26] D. Elad, D. Katz, E. Kimmel, S. Einav, Numerical schemes for unsteady fluid flow through collapsible tubes, *Journal of Biomedical Engineering* 13 (1) (1991) 10–18.
- [27] J.-M. Fullana, S. Zaleski, A branched one-dimensional model of vessel networks, *Journal of Fluid Mechanics* 621 (2009) 183–204.
- [28] J. Wan, B. Steele, S. A. Spicer, S. Strohsband, G. R. Feijó, T. J. Hughes, C. A. Taylor, A one-dimensional finite element method for simulation-based medical planning for cardiovascular disease, *Computer Methods in Biomechanics & Biomedical Engineering* 5 (3) (2002) 195–206.
- [29] S. J. Sherwin, L. Formaggia, J. Peiro, V. Franke, Computational modelling of 1d blood flow with variable mechanical properties and its application to the simulation of wave propagation in the human arterial system, *International Journal for Numerical Methods in Fluids* 43 (6-7) (2003) 673–700.
- [30] N. Cavallini, V. Caleffi, V. Coscia, Finite volume and weno scheme in one-dimensional vascular system modelling, *Computers & Mathematics with Applications* 56 (9) (2008) 2382–2397.
- [31] O. Delestre, P.-Y. Lagrée, A ‘well-balanced’ finite volume scheme for blood flow simulation, *International Journal for Numerical Methods in Fluids* 72 (2) (2013) 177–205.
- [32] K. S. Matthys, J. Alastruey, J. Peiró, A. W. Khir, P. Segers, P. R. Verdonck, K. H. Parker, S. J. Sherwin, Pulse wave propagation in a model human arterial network: assessment of 1-d numerical simulations against in vitro measurements, *Journal of Biomechanics* 40 (15) (2007) 3476–3486.
- [33] E. Marchandise, M. Willemet, V. Lacroix, A numerical hemodynamic tool for predictive vascular surgery, *Medical Engineering & Physics* 31 (1) (2009) 131–144.
- [34] J. Mynard, P. Nithiarasu, A 1d arterial blood flow model incorporating ventricular pressure, aortic valve and regional coronary flow using the locally conservative galerkin (lcg) method, *Communications in Numerical Methods in Engineering* 24 (5) (2008) 367–417.
- [35] X. Wang, 1d modeling of blood flow in networks: Numerical computing and applications, Ph.D. thesis, Paris 6 (2014).
- [36] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, A. Anandkumar, Fourier neural operator for parametric partial differential equations, *arXiv preprint arXiv:2010.08895* (2020).
- [37] L. Lu, X. Meng, Z. Mao, G. E. Karniadakis, Deepxde: A deep learning library for solving differential equations, *SIAM Review* 63 (1) (2021) 208–228.
- [38] B. Yu, et al., The deep ritz method: a deep learning-based numerical algorithm for solving variational problems, *Communications in Mathematics and Statistics* 6 (1) (2018) 1–12.
- [39] L. Lyu, Z. Zhang, M. Chen, J. Chen, MIM: A deep mixed residual method for solving high-order partial differential equations, *Journal of Computational Physics* 452 (2022) 110930.
- [40] J. Yang, Q. Zhu, A local deep learning method for solving high order partial differential equations, *Numerical Mathematics-Theory Methods and Applications* (2021).

- [41] Y. Lu, J. Lu, M. Wang, A priori generalization analysis of the deep ritz method for solving high dimensional elliptic partial differential equations, in: *Conference on Learning Theory*, PMLR, 2021, pp. 3196–3241.
- [42] J. Müller, M. Zeinhofer, Error estimates for the variational training of neural networks with boundary penalty, *arXiv preprint arXiv:2103.01007* (2021).
- [43] S. Mishra, R. Molinaro, Estimates on the generalization error of physics-informed neural networks for approximating a class of inverse problems for pdes, *IMA Journal of Numerical Analysis* 42 (2) (2022) 981–1022.
- [44] L. Li, X.-C. Tai, J. Yang, Generalization error analysis of neural networks with gradient based regularization, *Communications in Computational Physics* 32 (4) (2022) 1007–1038.
- [45] L. Li, X.-c. Tai, J. Yang, Q. Zhu, Priori error estimate of deep mixed residual method for elliptic pdes, *arXiv preprint arXiv:2206.07474* (2022).
- [46] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778.
- [47] L. Lu, X. Meng, S. Cai, Z. Mao, S. Goswami, Z. Zhang, G. E. Karniadakis, A comprehensive and fair comparison of two neural operators (with practical extensions) based on fair data, *Computer Methods in Applied Mechanics and Engineering* 393 (2022) 114778.
- [48] Z. Li, H. Zheng, N. Kovachki, D. Jin, H. Chen, B. Liu, K. Azizzadenesheli, A. Anandkumar, Physics-informed neural operator for learning partial differential equations, *arXiv preprint arXiv:2111.03794* (2021).
- [49] S. Goswami, M. Yin, Y. Yu, G. E. Karniadakis, A physics-informed variational deeponet for predicting crack path in quasi-brittle materials, *Computer Methods in Applied Mechanics and Engineering* 391 (2022) 114587.
- [50] N. Westerhof, J.-W. Lankhaar, B. E. Westerhof, The arterial windkessel, *Medical & Biological Engineering & Computing* 47 (2) (2009) 131–141.
- [51] J. Alastruey, K. H. Parker, S. J. Sherwin, et al., Arterial pulse wave haemodynamics, in: *11th International Conference on Pressure Surges*, Vol. 30, Virtual PiE Led t/a BHR Group Lisbon, Portugal, 2012, pp. 401–443.
- [52] A. Paviglianiti, V. Randazzo, S. Villata, G. Cirrincione, E. Pasero, A comparison of deep learning techniques for arterial blood pressure prediction, *Cognitive Computation* 14 (5) (2022) 1689–1710.
- [53] L. Lu, R. Pestourie, W. Yao, Z. Wang, F. Verdugo, S. G. Johnson, Physics-informed neural networks with hard constraints for inverse design, *SIAM Journal on Scientific Computing* 43 (6) (2021) B1105–B1132.
- [54] A. P. Avolio, L. M. Van Bortel, P. Boutouyrie, J. R. Cockcroft, C. M. McEniery, A. D. Protogerou, M. J. Roman, M. E. Safar, P. Segers, H. Smulyan, Role of pulse pressure amplification in arterial hypertension: experts’ opinion and review of the data, *Hypertension* 54 (2) (2009) 375–383.
- [55] Z. Gao, L. Yan, T. Zhou, Failure-informed adaptive sampling for pinns, *SIAM Journal on Scientific Computing* 45 (4) (2023) A1971–A1994.
- [56] A. E. Johnson, T. J. Pollard, L. Shen, L.-w. H. Lehman, M. Feng, M. Ghassemi, B. Moody, P. Szolovits, L. Anthony Celi, R. G. Mark, Mimic-iii, a freely accessible critical care database, *Scientific Data* 3 (1) (2016) 1–9.
- [57] M. Kachuee, M. Kiani, M. Hoda, M. Shabany, Cuff-Less Blood Pressure Estimation, *UCI Machine Learning Repository*, DOI: <https://doi.org/10.24432/C5B602> (2015).
- [58] J. Hong, J. Gao, Q. Liu, Y. Zhang, Y. Zheng, Deep learning model with individualized fine-tuning for dynamic and beat-to-beat blood pressure estimation, in: *2021 IEEE 17th International Conference on Wearable and Implantable Body Sensor Networks (BSN)*, IEEE, 2021, pp. 1–4.
- [59] P. H. Charlton, J. Mariscal Harana, S. Vennin, Y. Li, P. Chowienzyk, J. Alastruey, Modeling arterial pulse waves in healthy aging: a database for in silico evaluation of hemodynamics and pulse wave indexes, *American Journal of Physiology-Heart and Circulatory Physiology* 317 (5) (2019) H1062–H1085.
- [60] I. S. Association, et al., Ieee standard for wearable cuffless blood pressure measuring devices, *IEEE Std* (2014) 1708–2014.